RESUMEN

Chimpoco, es el nombre de un enorme y ambicioso proyecto, en el que el autor de este trabajo se adentró a finales de 2007 y que se prolonga en el tiempo hasta un indefinido futuro. Este proyecto consiste en el desarrollo de una versión personal y mejorada del famoso juego Pokémon, desarrollado por la empresa japonesa Nintendo.

"Chimpoco, desarrollo de un videojuego RPG basado en la librería SDL" es el título del trabajo de fin de grado que muestra una pequeña porción del futuro proyecto Chimpoco. Antes de comenzar, es necesario diferenciar entre Chimpoco como proyecto y Chimpoco como Trabajo de Fin de Grado.

Este Trabajo de Fin de Grado contiene unas pocas funcionalidades de todas las que compondrán la futura aplicación y supone un salto de calidad en el desarrollo del proyecto, ya que aplica todos los conocimientos adquiridos a lo largo de la carrera para comenzar a construir el futuro sistema.

Por lo tanto, en este trabajo, se comenzará a construir el proyecto Chimpoco, teniendo siempre presente que los objetivos van mucho más allá de los marcados para este trabajo. Por ello, muchas de las funcionalidades se definirán como funciones de desarrollo opcional, lo que permitirá dejar mucho trabajo analizado y preparado para el futuro, y facilitar la incorporación de nuevos módulos al sistema. Este trabajo opcional no viene sino a respaldar el desarrollo independiente del alumno que, en su tiempo libre, invertirá tiempo en el trabajo de forma paralela. Por supuesto, este desarrollo independiente no estará ligado ni a los objetivos ni a la planificación temporal de este trabajo de fin de grado, para así poder respetar el marco de desarrollo establecido por la Universidad, sin limitar las ganas e interés del alumno en hacer crecer su proyecto.

4. GESTIÓN DEL PROYECTO

4.1. INTRODUCCIÓN

Gestionar correctamente un proyecto es vital para garantizar tanto la calidad de sus procesos como del producto en sí mismo, así como para evitar su fracaso. En este proceso de gestión, se definirá la metodología a utilizar en el proyecto, se identificarán los diferentes riesgos que pueden afectar a su desarrollo, se creará la estructura de descomposición del trabajo que permitirá organizar y definir el alcance, se definirán los recursos humanos, económicos y materiales necesarios, a la vez que la planificación temporal, que permitirá organizar los recursos humanos, en función de la metodología escogida, para así satisfacer las restricciones temporales y económicas; y, finalmente, se establecerán los procesos que permitirán controlar los cambios en el proyecto y disponer, en todo momento, de una versión estable para cada elemento del mismo.

4.2. METODOLOGÍA UTILIZADA

Escoger una metodología que se ajuste correctamente al proyecto que se está desarrollando, es una de las principales y más importantes tareas en su gestión. Un ciclo de vida es una estructura que, aplicada adecuadamente al desarrollo de un producto software, permitirá estructurar, planificar y controlar el proceso de desarrollo. Se debe tener presente que una buena elección de un ciclo de vida optimizará el proceso de desarrollo, mientras que una mala elección condenará el proyecto al fracaso.

Conociendo la importancia de esta decisión, la metodología de desarrollo escogida, para este proyecto, es el modelo de ciclo de vida incremental. En este modelo el producto se desarrolla en diferentes incrementos, al final de los cuales, se obtienen cada uno de los módulos funcionales del software. La integración de estos módulos permite obtener el software final. Este enfoque de desarrollo parece idóneo para un proyecto en el que, sus diferentes módulos, son bastante independientes y donde se pretende, en un futuro, integrar nuevas funcionalidades. Cada una de estas funcionalidades puede ser tratada como un módulo independiente, e ir integrándose al producto final, incremento tras incremento. Esto permite que el cliente reciba cada uno de los incrementos sin tener que esperar a una hipotética integración final, lo cual, en este proyecto, facilitará un producto estable a los usuarios que se irá, poco a poco, actualizando con nuevas funcionalidades. Se debe tener presente que, por lo tanto, cada una de estas iteraciones posee su propia fase de diseño, implementación y pruebas.

El fundamento de este producto, sobre todo en el futuro, es la de proporcionar a los usuarios nuevas funcionalidades de forma relativamente rápida, a la vez que los contenidos del producto se hacen más profundos (más personajes, más regiones, más Pokémon…), siendo así un producto en continua evolución. Teniendo esto presente, se ha considerado útil incorporar ciertas características de las metodologías ágiles, concretamente de Scrum. Este tipo de ciclos de vida permiten que cada incremento o iteración, de duración entre una y cuatro semanas, agregue pequeñas funcionalidades al software, primando un desarrollo veloz y evaluando, al comienzo de cada iteración, las prioridades del producto. También permiten reducir los riesgos inherentes al proyecto dividiéndolo en segmentos más pequeños y facilitando los cambios en el proceso de desarrollo.

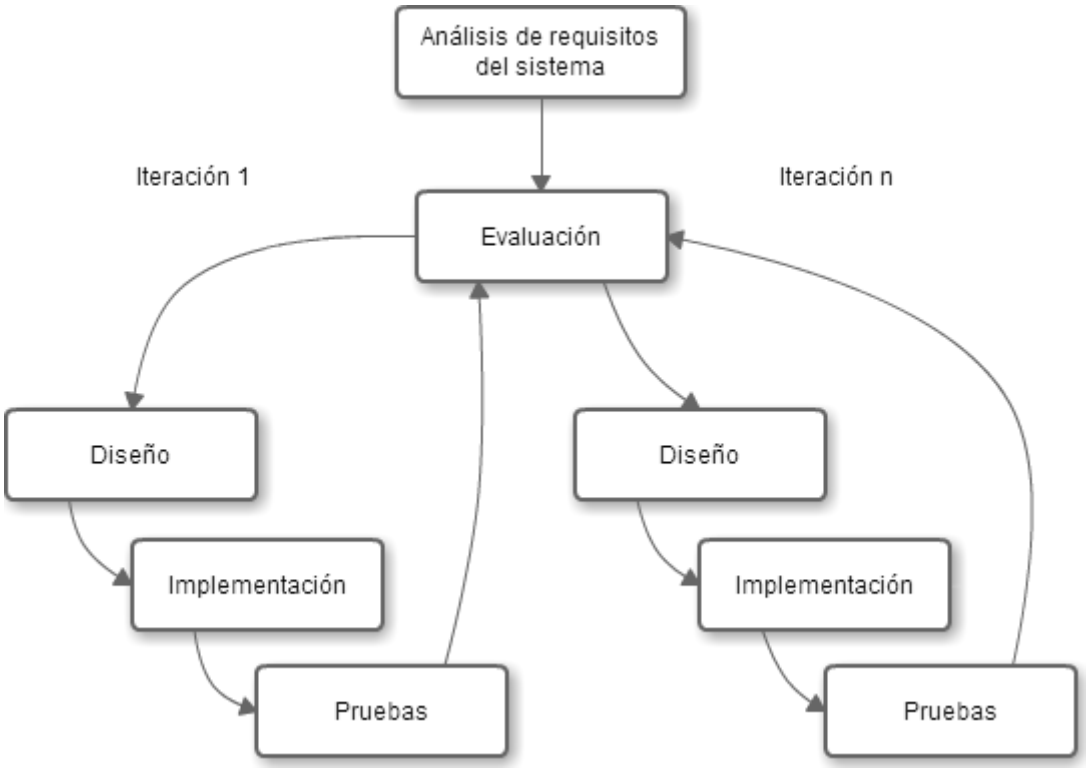

Figura 4.1. Metodología de desarrollo

Para este proyecto se ha dividido el desarrollo en cuatro incrementos, respetando la importancia y urgencia de los requisitos identificados, abarcando cada uno de los grandes apartados de este proyecto.

4.2.1. Análisis del alcance de cada iteración

En el **primer incremento** (*véase* Tabla 4.1) se desarrollará el apartado gráfico de la aplicación: el software necesario para gestionar la interfaz gráfica y dibujar sobre ella. Además, incorporará las funcionalidades de gestión de audio.

Iteración 1	
Descripción	Gestionar la interfaz gráfica y el audio
Requisitos que debe cumplir	• UC-RS-01 *Aumentar el zoom* • UC-RS-02 *Reducir el zoom* • UC-AS-02 *Pausar y reanudar audio* • FRQ-01 *Crear ventana* • FRQ-02 *Cargar texturas* • FRQ-03 *Zoom* • FRQ-04 *Escribir en la ventana* • FRQ-05 *Cargar y reproducir archivos de audio* • NFR-DRS-01 *Utilización de SDL* • NFR-IRQ-01 *Tamaño de interfaces* • NFR-IRQ-02 *Ampliar pantalla* • NFR-ORQ-01 *Sistema modular* • NFR-PRQ-01 *Carga inicial de datos* • NFR-PRQ-02 *Estabilidad de la memoria utilizada*

Tabla 4.1. Alcance de la primera iteración

En la **segunda iteración** (*véase* Tabla 4.2), la mayor de todas tanto en tamaño como en complejidad, se abarcará la funcionalidad total del Mundo Pokémon, desde la creación de mapas al movimiento de personajes, y la gestión de la partida.

Iteración 2	
Descripción	Modo mundo y gestión de la partida
Requisitos que debe cumplir	• UC-GS-01 *Iniciar partida* • UC-GS-02 *Continuar partida* • UC-GS-03 *Guardar partida* • UC-WS-01 *Mover al protagonista* • UC-WS-02 *Acceder a una entrada* • UC-WS-03 Acceder a un mapa fronterizo • UC-WS-04 *Recoger objeto olvidado* • UC-WS-05 *Dialogar con un personaje* • UC-WS-06 *Dialogar con una estructura* • UC-WS-07 *Mover una estructura* • UC-WS-08 *Romper una estructura* • FRQ-07 *Cargar mapa* • FRQ-08 *Modificar mapa* • FRQ-09 *Activar evento de historia por mapa* • FRQ-10 *Calcular movimiento de personajes* • FRQ-19 *Calcular colisiones* • FRQ-23 *Recibir Pokémon* • FRQ-24 *Recibir objeto* • NFR-RRQ-01 *Garantizar la atomicidad al guardar la partida*

Tabla 4.2. Alcance de la segunda iteración

En la **tercera iteración** se desarrollarán las funcionalidades derivadas del Modo Combate contempladas en la Tabla 4.3.

Iteración 3	
Descripción	Combate Pokémon
Requisitos que debe cumplir	• UC-BS-01 *Huir del combate* • UC-BS-02 *Ver lista de movimientos* • UC-BS-03 *Usar un movimiento* • UC-BS-04 *Evolucionar a un Pokémon* • UC-BS-05 *Aprender un movimiento nuevo* • UC-BS-06 *Olvidar un movimiento* • FRQ-11 *Calcular huida* • FRQ-12 *Calcular daño* • FRQ-13 *Calcular movimiento del Pokémon rival* • FRQ-14 *Calcular movimientos para Pokémon salvaje* • FRQ-15 *Subir de nivel* • FRQ-16 *Calcular encuentro Pokémon* • FRQ-17 *Calcular PID de Pokémon salvaje* • FRQ-18 *Calcular captura Pokémon* • FRQ-21 *Luchar con un personaje Pokémon* • FRQ-22 *Iniciar combate Pokémon*

Tabla 4.3. Alcance de la tercera iteración

Y, finalmente, en el cuarto y **último incremento** (*véase* Tabla 4.4), se desarrollarán todas las funciones que se han considero como opcionales, es decir, todos los requisitos restantes que se han definido con una importancia "Quedaría bien". Por lo tanto, se trata de un incremento opcional en el que se abarcarán funcionalidades que extiendan el tiempo de trabajo de este proyecto de fin de grado, es decir, dichas funcionalidades se podrán desarrollar en el tiempo libre del autor, nunca supeditado a la planificación temporal de este trabajo. Se ha considerado de esta manera, pues es un incremento que, tarde o temprano, el autor desarrollará y que resulta interesante reflejar en este trabajo, ya que es viable la inclusión de alguna funcionalidad. En ningún caso, por lo tanto, se exigirá el cumplimiento de cualquiera de los requisitos asociados a esta iteración, sino que será tratado como un aliciente.

Iteración 4	
Descripción	Interfaces de Pokédex, equipo y mapa
Requisitos que debe cumplir	• UC-AS-01 *Modificar volumen* • UC-PS-01 *Abrir Pokédex* • UC-PS-02 *Navegar por la Pokédex* • UC-PS-03 *Ver datos de especie* • UC-TS-01 *Abrir menú de equipo* • UC-TS-02 *Seleccionar un Pokémon* • UC-TS-03 *Teletransporte* • UC-TS-04 *Excavar* • UC-TS-05 *Cambiar la posición de un Pokémon* • UC-TS-06 *Ver información de un Pokémon* • UC-TS-07 *Ver los movimientos de un Pokémon* • UC-TS-08 *Ver las estadísticas de un Pokémon* • UC-TS-09 *Seleccionar un movimiento* • UC-TS-10 *Cambiar un movimiento* • UC-IS-01 *Abrir mochila* • UC-IS-02 *Buscar en la mochila* • UC-IS-03 *Seleccionar objeto* • UC-IS-04 *Usar poción* • UC-IS-05 *Usar Pokéball* • UC-IS-06 *Tirar objeto* • UC-IS-07 *Añadir Pokémon al equipo* • UC-IS-08 *Liberar Pokémon* • UC-MS-01 *Abrir mapa* • UC-MS-02 *Volar a un lugar* • UC-MS-03 *Cerrar mapa* • UC-MS-04 *Navegar por el mapa* • FRQ-20 *Curar equipo Pokémon*

Tabla 4.4. Alcance de la cuarta iteración

Hay ciertos requisitos no funcionales que vienen derivados de buenas prácticas de desarrollo software, por lo que estos requisitos no están ligados a una iteración en concreto sino que lo están a todo el proyecto en sí. Estos requisitos que estarán presentes en todas las iteraciones son:

- NFR-RRQ-02 *Tolerancia a fallos*
- NFR-RRQ-03 *Fichero de log*
- NFR-URQ-01 *Interfaces similares a las de Nintendo*
- NFR-URQ-02 *Colores de texto*
- NFR-PRQ-01 *Carga inicial de datos*
- NFR-PRQ-02 *Estabilidad de la memoria utilizada*

- NFR-PRQ-03 *Velocidad de juego estable*
- NFR-PRQ-04 *Velocidad de operaciones complejas*
- NFR-ERQ-01 *Aplicación multilenguaje*
- NFR-ERQ-02 *Nuevas generaciones*
- NFR-SRQ-01 *Desbordamiento de buffer*
- NFR-SRQ-03 *Fugas de memoria*NFR-ORQ-01 *Sistema modular*

Se puede comprobar en el documento anexo RQA_Asignación_de_Requisitos.pdf, cómo todos los requisitos se hayan contemplados en, al menos, una iteración.

4.3. GESTIÓN DE RIESGOS

La gestión de riesgos es un proceso que permite identificar las vulnerabilidades y amenazas a las que los diferentes elementos del proyecto se encuentran expuestos, y analizar su probabilidad de ocurrencia e impacto sobre ellos con la finalidad de establecer controles adecuados para evitar la ocurrencia de dichos riesgos y, en caso de que ocurran, disminuir su impacto.

De acuerdo con [PMI08], la gestión de riesgos comprenderá cinco etapas:

- **Planificación de la gestión de riesgos**: Se definirán los valores de impacto y probabilidad que permitirán, más adelante, analizar y priorizar los riesgos mediante una escala de impacto; y se establecerá la frecuencia con la que se realizará la gestión de riesgos durante el ciclo de vida del proyecto.
- **Identificación de riesgos**: Se identificarán los diferentes riesgos inherentes al proyecto mediante tormenta de ideas y análisis de asunciones.
- **Análisis de riesgos**: Se realizará un análisis cuantitativo y cualitativo de todos los riesgos identificados, valorando la prioridad en función de su probabilidad de ocurrencia y consecuencias de su materialización.
- **Planificación de la respuesta a los riesgos**: Se definirán los planes de contingencia que permitirán evitar y minimizar el impacto de cada uno de los riesgos.
- **Seguimiento y control de riesgos**: Permitirá supervisar continuamente el proyecto para detectar riesgos nuevos o cambiantes.

En primer lugar, se definirán los valores de impacto y probabilidad (*véase* Tabla 4.5), la escala de impacto y la matriz de probabilidad e impacto. Finalmente, se presentarán los riesgos identificados, analizados y planificados. La revisión de los riesgos se realizará con una frecuencia de una revisión por cada iteración, realizando ésta siempre al inicio de cada iteración.

Probabilidad		Impacto
Muy baja	Menor que 5%	Insignificante
Baja	Entre 5% y 10%	Pequeño
Moderada	Entre 10% y 20%	Tolerable
Alta	Entre 20% y 40%	Serio
Muy alta	Mayor que 40%	Catastrófico

Tabla 4.5. Definición de valores de impacto y probabilidad

5.1.2. PAQUETE CONTROLLER

Primero, se presenta el diagrama de clases para el paquete Controller (*véase* Figura 5.4).

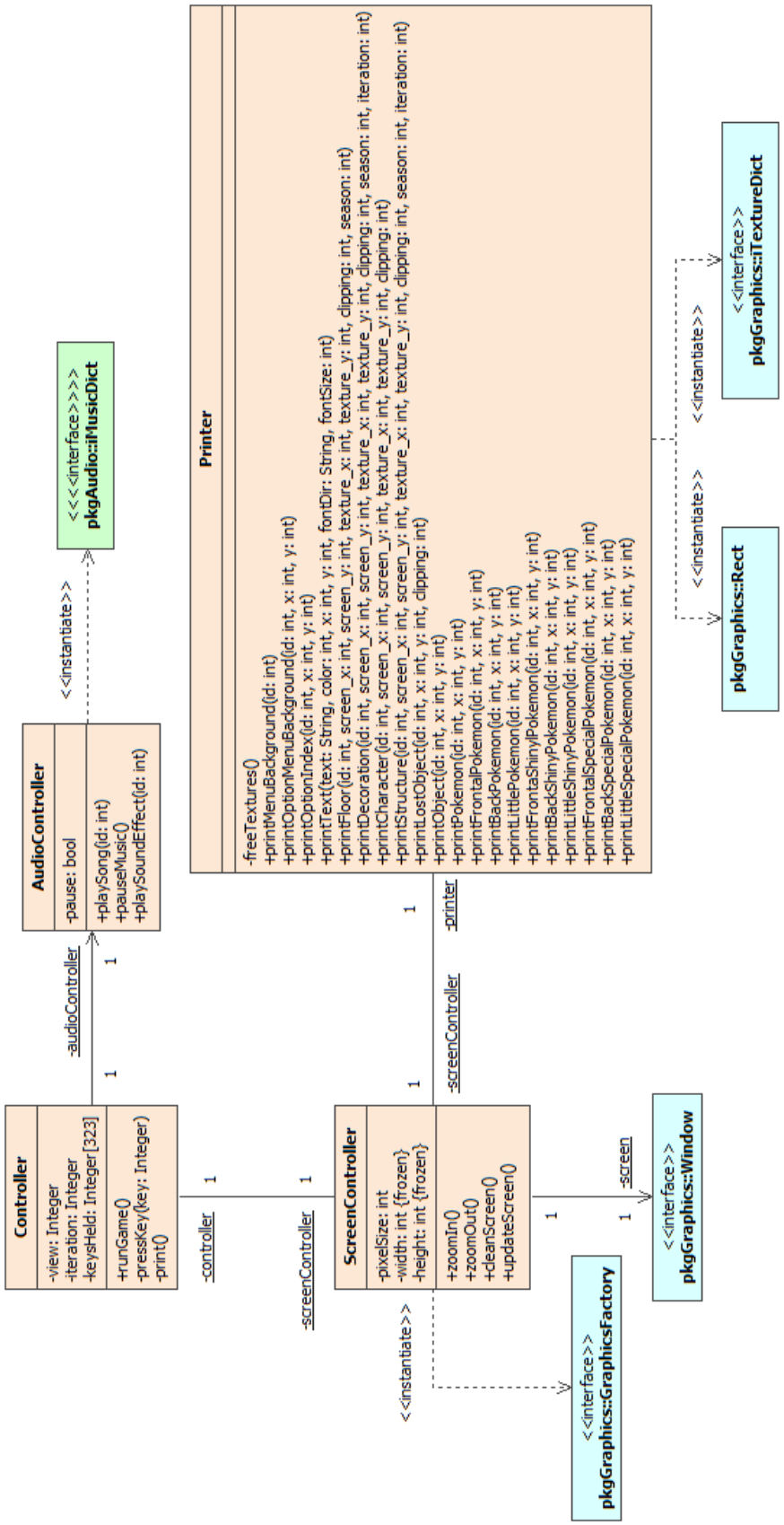

Figura 5.4. Diagrama de clases del paquete Controller

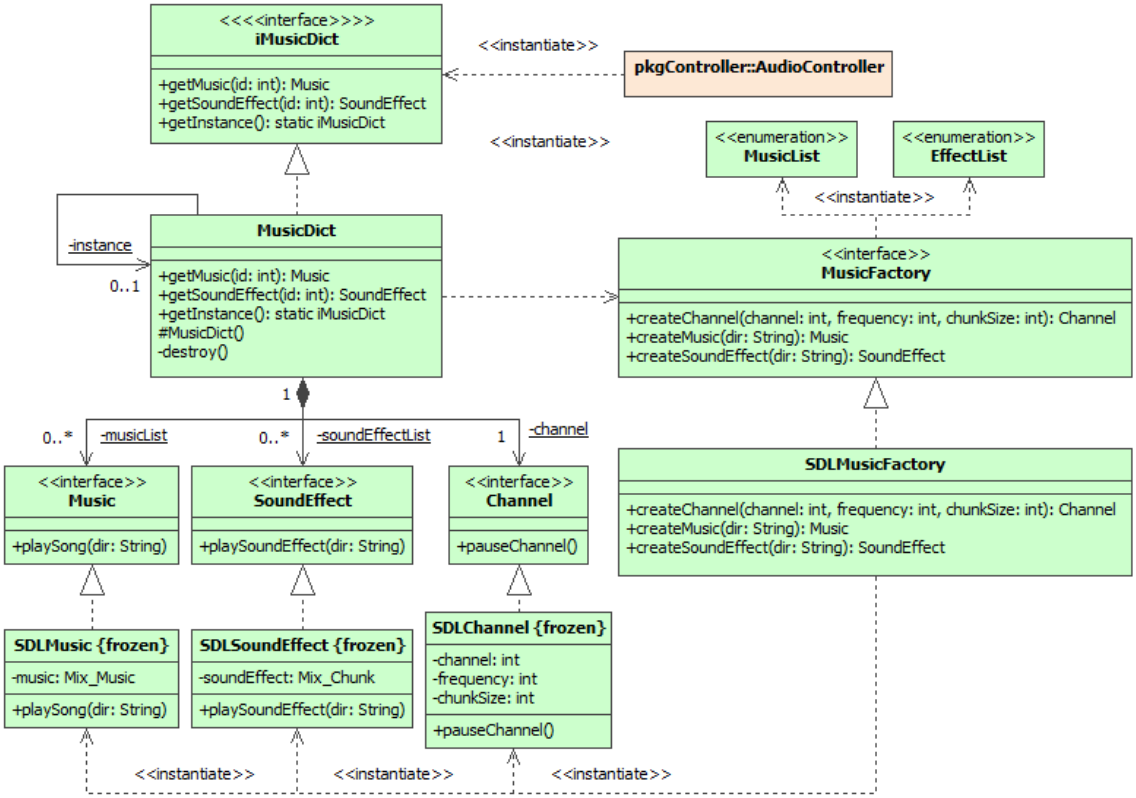

Figura 5.5. Diagrama de clases del paquete Audio

A continuación, se presentan las diferentes tarjetas CRC generadas para las clases que conforman este paquete:

iMusicDict	
Responsabilidad	**Colaboración**
Proporcionar un punto de acceso para la obtención de cada uno de los componentes de audio	

Tabla 5.6. Tarjeta CRC de la clase iMusicDict

MusicDict	
Responsabilidad	**Colaboración**
Mantener referencias a los diferentes componentes de audio de la aplicación	Music SoundEffect Channel
Transferir la referencia del componente de audio solicitado	Music SoundEffect Channel
Crear y liberar los componentes de audio almacenados durante la ejecución	Music SoundEffect Channel

Tabla 5.7. Tarjeta CRC de la clase MusicDict

Music	
Responsabilidad	**Colaboración**
Proporcionar el conjunto de métodos realizables con los archivos de audio de forma independiente de la tecnología utilizada para su implementación	

5.1.4. PAQUETE GRAPHICS

Primero, se presenta el diagrama de clases para el paquete Graphics (véase Figura 5.6).

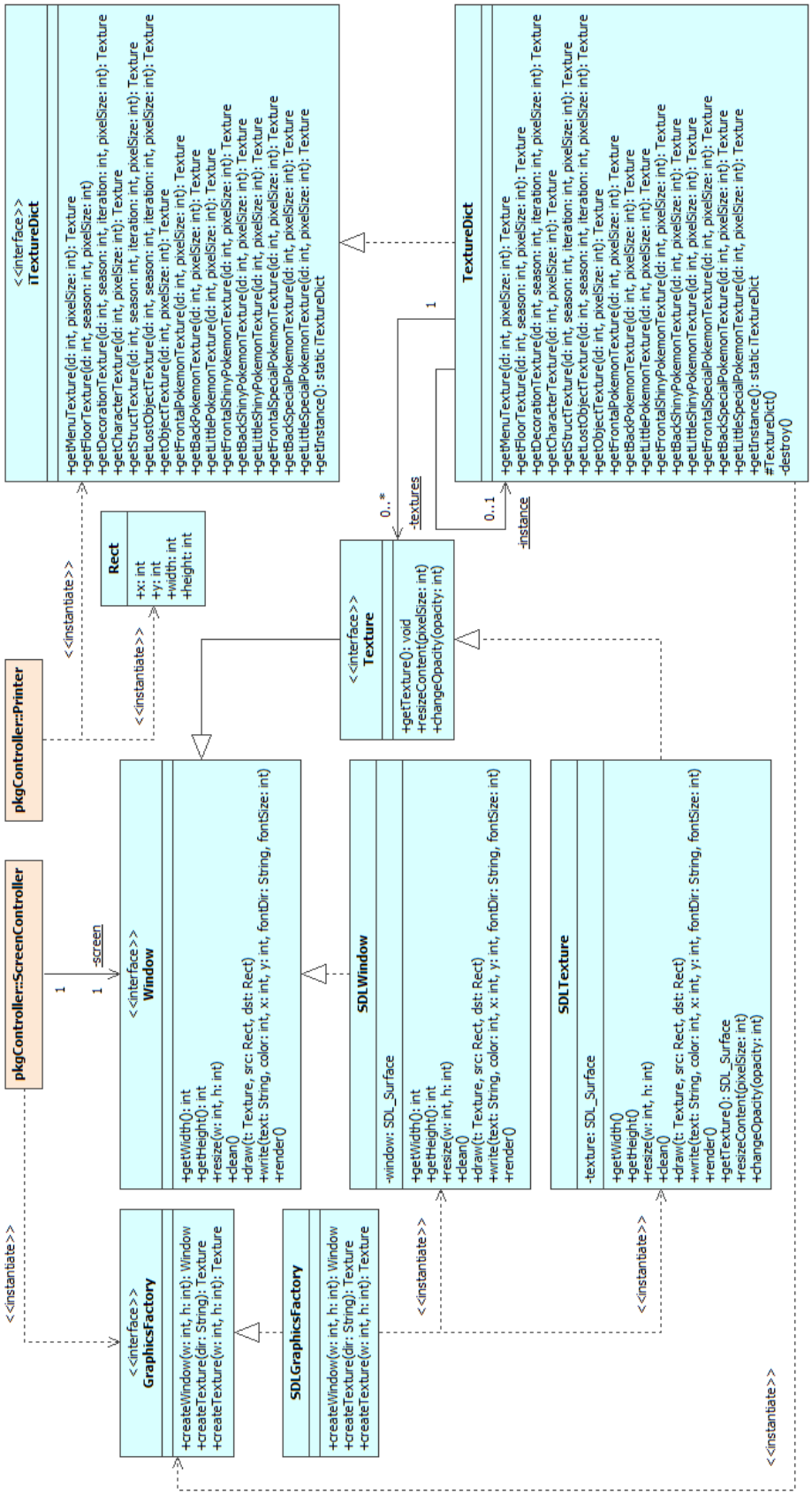

Figura 5.6. Diagrama de clases para el paquete Graphics

6. Desarrollo: Segunda iteración

Tal y como ya se ha adelantado en el apartado 4.2.1, en esta fase se mostrarán los aspectos de diseño, implementación y pruebas relativos al desarrollo de mapas y la gestión de la partida. Al final de este incremento, se deberá disponer de una aplicación que permita al usuario recorrer diferentes mapas, interactuar con personajes, objetos y estructuras; y empezar, cargar y guardar su partida. Siguiendo el esquema detallado en la Estructura de Descomposición del Trabajo (véase apartado 4.4), se comenzará ésta iteración con una fase de evaluación y análisis. De la misma forma que se detalla en el anterior capítulo, una vez revisados los requisitos y los riesgos del proyecto, se ha decidido no crear ningún apartado para dicha fase, pues no se ha encontrado ninguna necesidad de modificar o añadir riesgos o requisitos asociados.

6.1. Diseño

Para este incremento se han diseñado una serie de clases que, agrupadas en los paquetes Controller, World y Game (controlador, mundo y partida), permiten gestionar la partida y los diferentes escenarios del juego. Para describir la estructura del sistema, se utilizarán diagramas de clases y, para mostrar las dependencias entre los diferentes paquetes, un diagrama de paquetes.

6.1.1. Diagrama de paquetes

Como se puede observar en el diagrama de la Figura 6.1, debido a su nueva complejidad, el paquete Controller se ha dividido a su vez en tres paquetes: Controllers, Helpers y MoveStrategy. Todas las clases previamente presentes en Controller se mantienen en Controllers, a excepción de Printer, que se ha añadido al paquete Helpers.

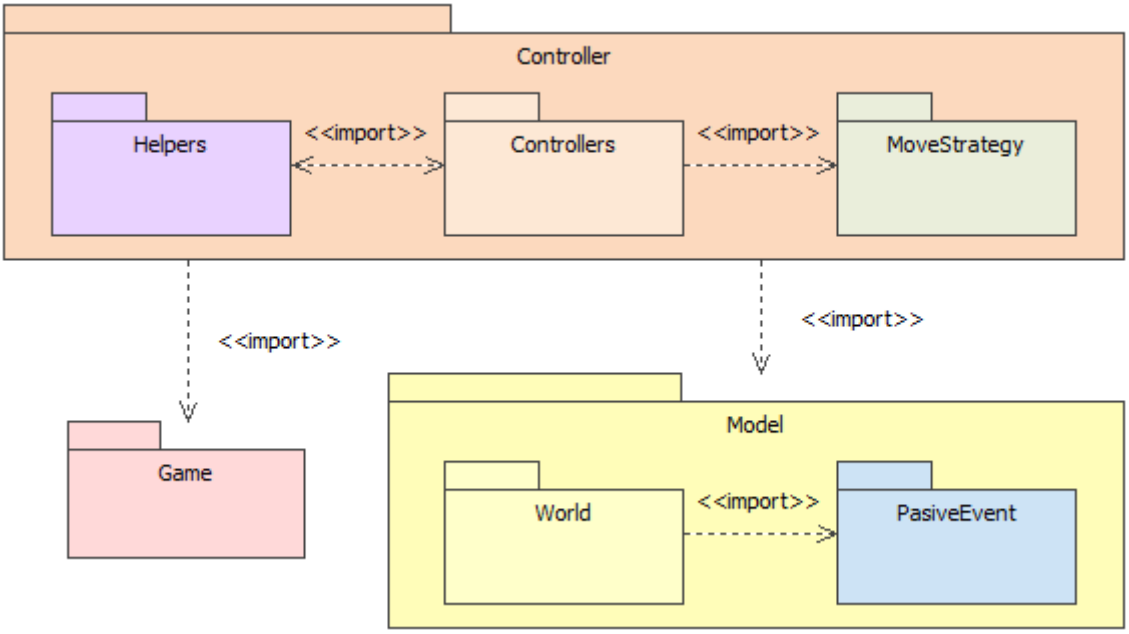

Figura 6.1. Diagrama de paquetes – Segunda iteración

El paquete Controllers contiene ahora todos los controladores y Helpers todas las clases de ayuda de los controladores. El paquete MoveStrategy contiene todas las estrategias de movimiento de personajes.

El paquete Model contiene todas las clases que conforman el Modelo (*véase* 5.1.1) y, por lo tanto, son clases meramente contenedoras. Dicho paquete también se ha subdividido en los paquetes World y PasiveEvent, que contienen los diferentes elementos del mundo Pokémon y los eventos pasivos de juego, respectivamente.

Tanto PasiveEvent como MovementStrategy son clases y subclases muy susceptibles a ampliación, por lo que se ha optado por incluirlas en paquetes independientes y así facilitar el cambio.

6.1.2. Paquete Controller

Paquete Controllers

Primero, se presenta el diagrama de clases para el paquete Controller (*véase* Figura 6.2). Este paquete ya ha sido especificado en la primera iteración por lo que no se mostrarán ni explicarán las responsabilidades de las clases previamente existentes.

A continuación, utilizando la plantilla contenida en la Tabla 5.1, se presentan las diferentes tarjetas CRC generadas para las clases que conforman este paquete. Se debe tener en cuenta que la clase Controller de este mismo paquete, que antes colaboraba con ScreenController y AudioController para satisfacer sus responsabilidades, lo hace de la misma forma con WorldController, GameController e InitMenuController.

Existen enumeraciones en este paquete, igual que sucedía en el módulo anterior. Sus responsabilidades son las mismas: proveer identificadores a los objetos que dependen de ellas.

WorldController	
Responsabilidad	**Colaboración**
Inicializar los mapas y el protagonista en función del contenido de la partida	GameController iMapDict
Gestionar las funcionalidades presentes en los mapas	MapConstructor MovementStrategy
Contener la lógica de presentación de la interfaz *modo mundo*	Printer

Tabla 6.1. Tarjeta CRC de la clase WorldController

GameController	
Responsabilidad	**Colaboración**
Gestionar las funcionalidades de la partida: empezar, cargar y guardar	WorldController

Tabla 6.2. Tarjeta CRC de la clase GameController

InitMenuController	
Responsabilidad	**Colaboración**
Gestionar las funcionalidades del menú principal	
Contener la lógica de presentación del menú inicial del juego	Printer

Tabla 6.3. Tarjeta CRC de la clase InitMenuController

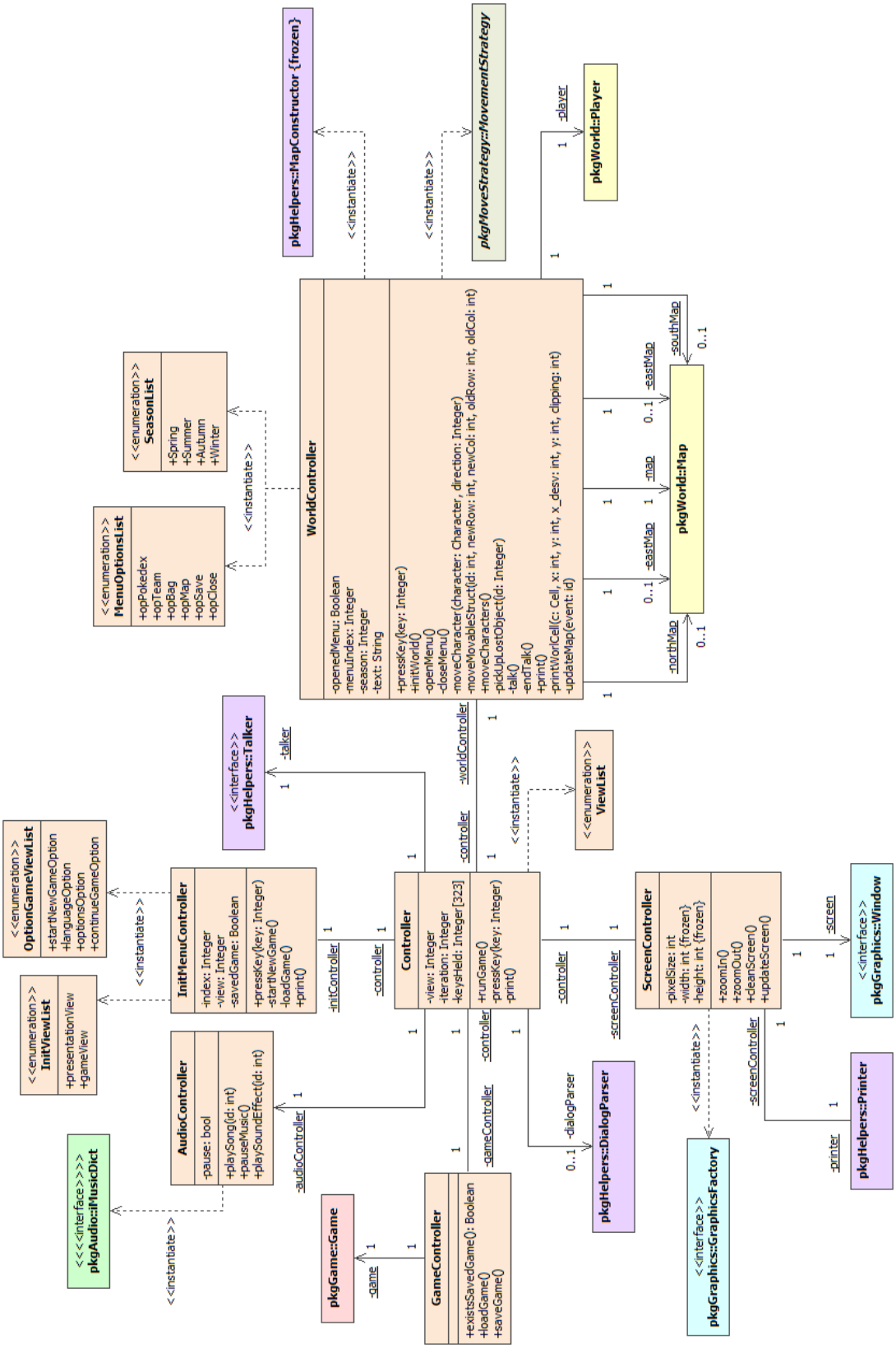

Figura 6.2. Diagrama de clases del paquete Controllers

Paquete Helpers

Se puede observar el diagrama de clases para este paquete en la Figura 6.3. Debido a su gran tamaño, se ha creído conveniente no mostrar, para la clase Printer, ni sus operaciones ni sus atributos en el diagrama, ya que, al no haber sido modificada, no aportaría nueva información al modelo.

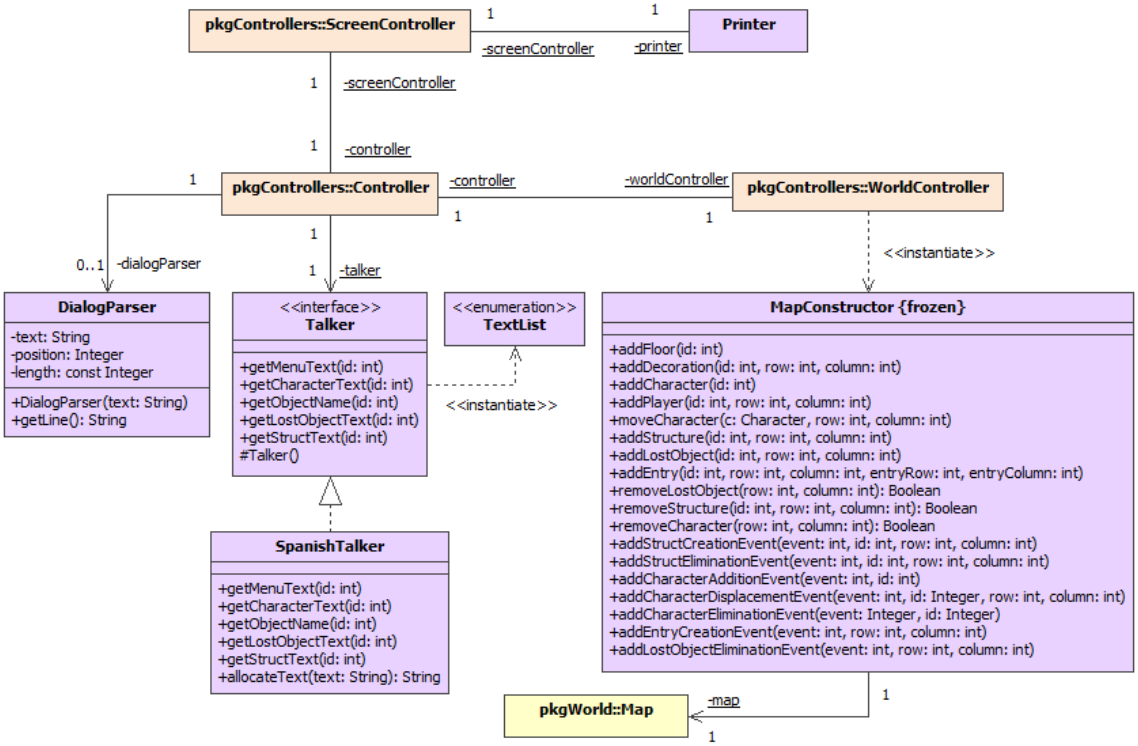

Figura 6.3. Diagrama de clases del paquete Helpers

Como sucedía anteriormente con, por ejemplo, Controller, la clase Printer, ya especificada, no será descrita de nuevo en este incremento.

MapConstructor	
Responsabilidad	**Colaboración**
Añadir, modificar y eliminar elementos a un mapa determinado	iFloorDict iDecorationDict iStructDict iObjectDict iCharacterDict iAppearenceDict

Tabla 6.4. Tarjeta CRC de la clase MapConstructor

Talker	
Responsabilidad	**Colaboración**
Proporcionar un punto de acceso para la obtención de cada uno de los diálogos de la aplicación	SpanishTalker

Tabla 6.5. Tarjeta CRC de la clase Talker

SpanishTalker	
Responsabilidad	**Colaboración**
Almacenar y proveer los diálogos de la aplicación en idioma español	

Tabla 6.6. Tarjeta CRC de la clase SpanishTalker

DialogParser	
Responsabilidad	**Colaboración**
Dividir un texto en segmentos cuyo tamaño máximo sea el de pantalla	

Tabla 6.7. Tarjeta CRC de la clase DialogParser

Paquete MoveStrategy

En primer lugar se muestra, en la Figura 6.4, el diagrama de clases de este paquete.

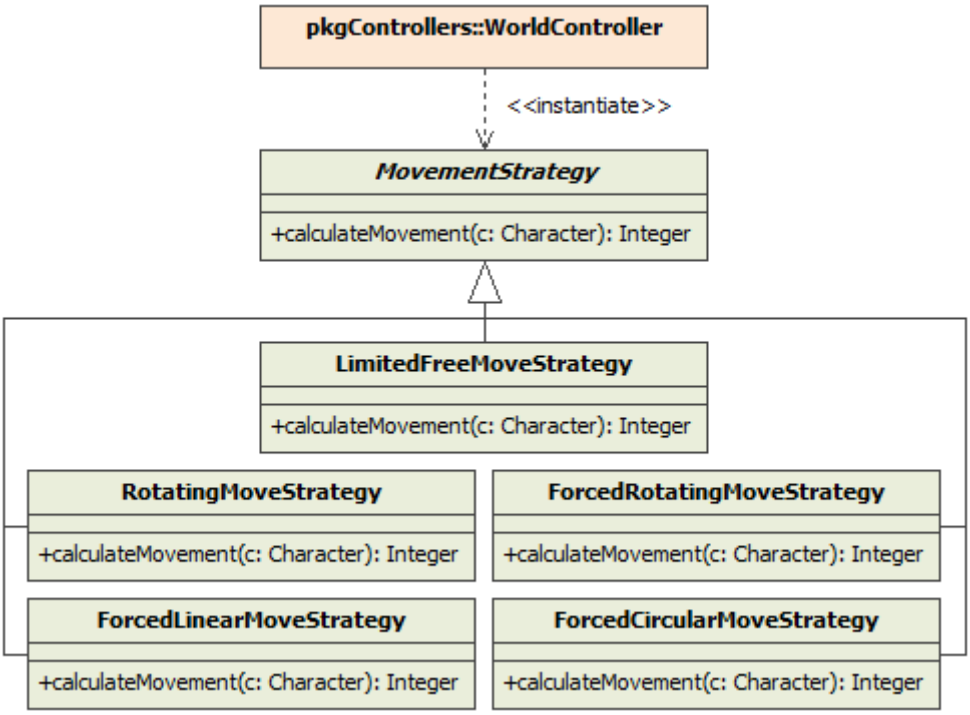

Figura 6.4. Diagrama de clases del paquete MoveStrategy

En segundo lugar, se listan las tarjetas CRC que muestran las responsabilidades de cada clase y sus colaboraciones para satisfacer dicha necesidad.

MovementStrategy	
Responsabilidad	**Colaboración**
Proporcionar un punto de acceso para el cálculo de movimiento de personajes	LimitedFreeMoveStrategy RotatingMoveStrategy ForcedRotatingMoveStrategy ForcedLinearMoveStrategy ForcedCircularMoveStrategy

Tabla 6.8. Tarjeta CRC de la clase MovementStrategy

LimitedFreeMoveStrategy	
Responsabilidad	**Colaboración**
Calcular el siguiente movimiento de un personaje dado siguiendo una estrategia de movimiento *libre limitado*	

Tabla 6.9. Tarjeta CRC de la clase LimitedFreeMoveStrategy

RotatingMoveStrategy	
Responsabilidad	**Colaboración**
Calcular el siguiente movimiento de un personaje dado siguiendo una estrategia de movimiento *giratorio*	

Tabla 6.10. Tarjeta CRC de la clase RotatingMoveStrategy

ForcedRotatingMoveStrategy	
Responsabilidad	**Colaboración**
Calcular el siguiente movimiento de un personaje dado siguiendo una estrategia de movimiento *giratorio obligado*	

Tabla 6.11. Tarjeta CRC de la clase ForcedRotatingMoveStrategy

ForcedLinearMoveStrategy	
Responsabilidad	**Colaboración**
Calcular el siguiente movimiento de un personaje dado siguiendo una estrategia de movimiento *obligado lineal*	

Tabla 6.12. Tarjeta CRC de la clase ForcedLinearMoveStrategy

ForcedCircularMoveStrategy	
Responsabilidad	**Colaboración**
Calcular el siguiente movimiento de un personaje dado siguiendo una estrategia de movimiento *obligado circular*	

Tabla 6.13. Tarjeta CRC de la clase ForcedCircularMoveStrategy

6.1.3. Paquete Game

En la Figura 6.5 se puede ver el diagrama de clases para el paquete Game. A continuación, se presenta la tarjeta CRC de la única clase que, por el momento, conforma este paquete:

Game	
Responsabilidad	**Colaboración**
Contener la información de la partida	

Tabla 6.14. Tarjeta CRC de la clase Game

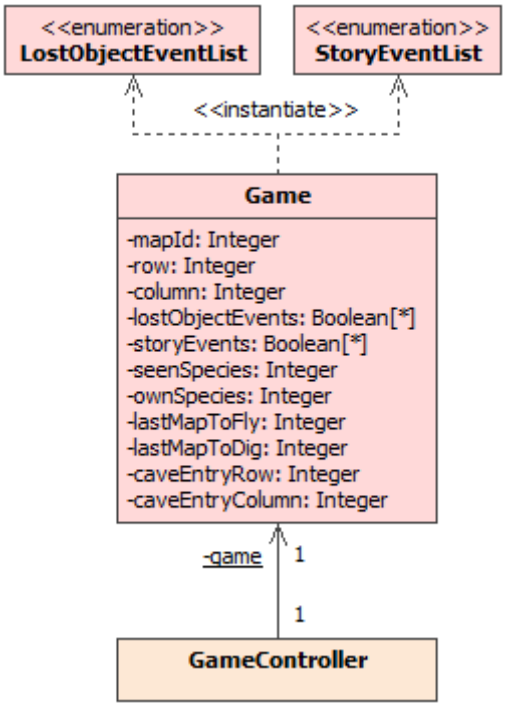

Figura 6.5. Diagrama de clases del paquete Game

6.1.4. PAQUETE MODEL

Paquete World

El paquete World es el más grande de este proyecto y contiene la mayor parte del Modelo de la aplicación y, por este motivo, se recomienda consultarlo en el soporte magnético.

En el diagrama de clases de la Figura 6.6 se han utilizado cuatro colores diferentes para facilitar la visión del sistema. Cada color identifica a un "tipo" de clase: gris a las enumeraciones, verde a las interfaces de los diccionarios, azul a las implementaciones de los diccionarios y amarillo al resto del modelo. Por lo tanto, dadas las dimensiones de este paquete y, a que todas las clases del mismo "tipo" tienen las mismas responsabilidades, se ha decidido hacer sólo una tarjeta CRC por cada uno de estos grupos de clases, entendiendo que el propio nombre de la clase y el diagrama son suficientes para comprender con qué tipo de datos trabaja para satisfacer su responsabilidad.

Enumeraciones	
Responsabilidad	**Colaboración**
Almacenar y proveer los identificadores de los diferentes elementos de mapa	

Tabla 6.15. Tarjeta CRC común a todas las enumeraciones del paquete World

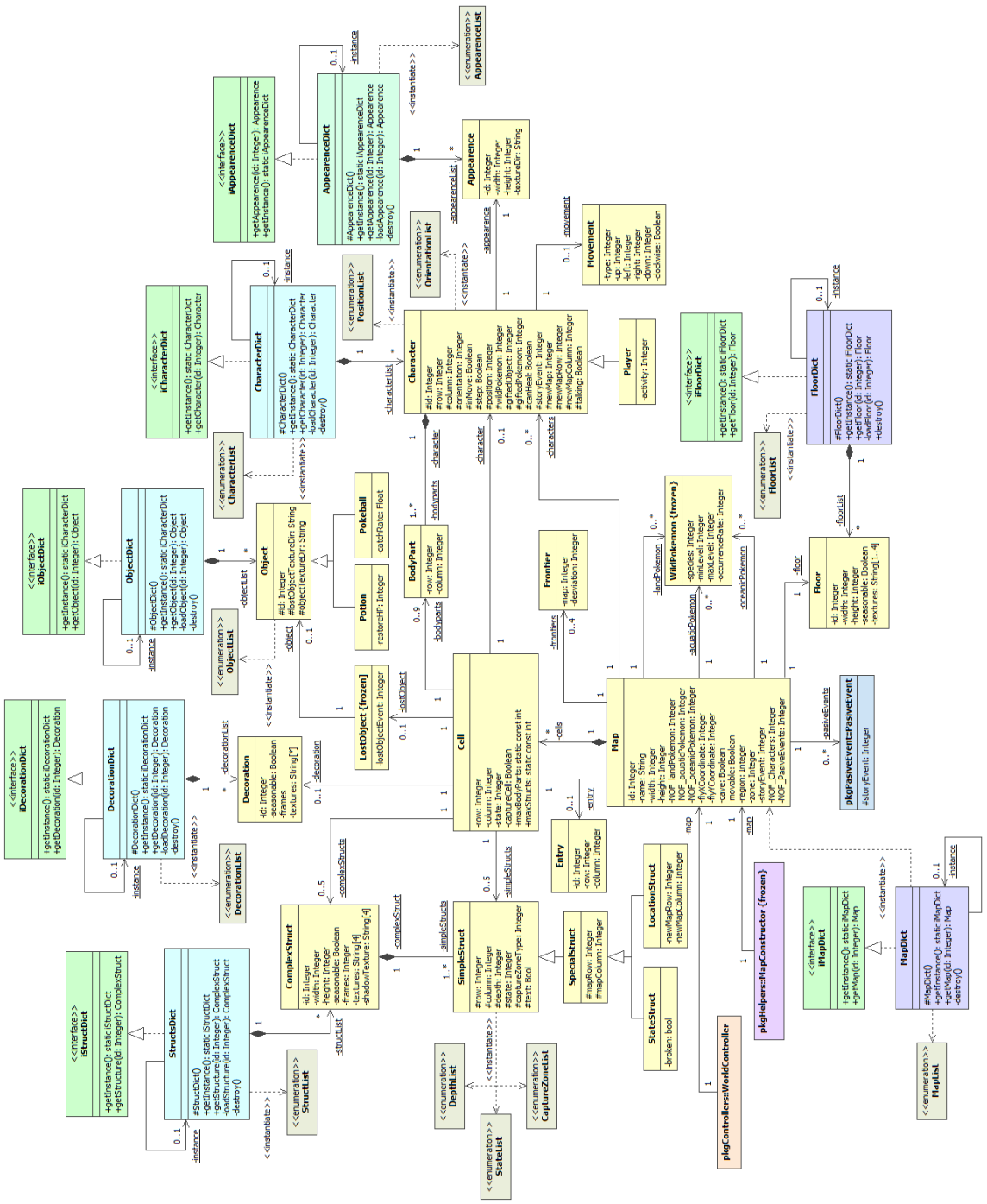

Figura 6.6. Diagrama de clases del paquete World

Interfaces de diccionario	
Responsabilidad	**Colaboración**
Proporcionar un punto de acceso para la obtención de cada uno de los elementos contenidos en los diccionarios	

Tabla 6.16. Tarjeta CRC común a todas las interfaces del paquete World

Diccionarios de datos	
Responsabilidad	**Colaboración**
Almacenar y proveer referencias a los elementos presentes en los mapas	

Tabla 6.17. Tarjeta CRC común a todos los diccionarios del paquete World

Clases de Modelo	
Responsabilidad	**Colaboración**
Contener la estructura de los mapas y sus elementos	

Tabla 6.18. Tarjeta CRC común a todas las clases de Modelo del paquete World

Paquete PasiveEvent

Para este paquete sólo se mostrará el diagrama de clases (véase Figura 6.7), pues las tarjetas CRC para cada una de las clases contenedoras es la misma que la usada en Tabla 6.18. Tarjeta CRC común a todas las clases de Modelo del paquete World.

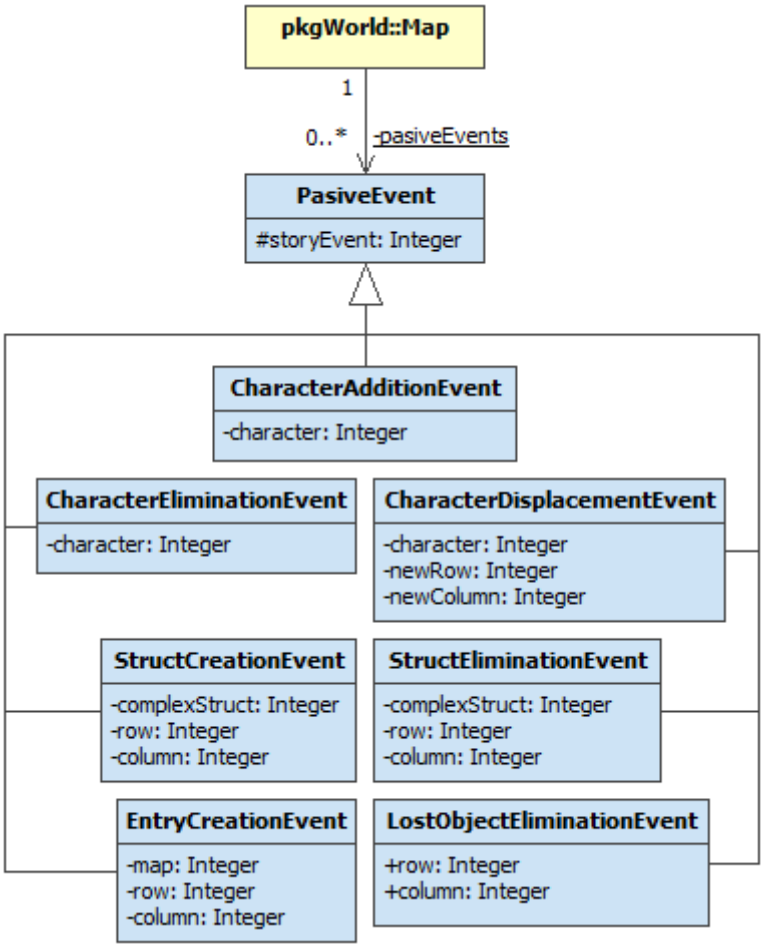

Figura 6.7. Diagrama de clases para el paquete PasiveEvent

6.1.5. PATRONES DE DISEÑO

La mayor parte de los patrones aplicados ya han sido explicados en el apartado correspondiente de la iteración anterior (*véase* apartado 5.1.5) por lo que sólo se explicará el funcionamiento del nuevo patrón utilizado:

- **Strategy**: Este patrón de comportamiento define y encapsula una familia de algoritmos que pueden ser intercambiables entre ellos. Es comúnmente utilizado en situaciones en las que se dispone de diferentes algoritmos para realizar una acción y que, en función de ciertos parámetros, se decidirá cuál de ellos utilizar. Cuenta con la ventaja de reducir notablemente la cantidad de código de una clase que, de otro modo, dispondría de una serie de cláusulas *if* con todos los algoritmos cargados al mismo tiempo de los cuales, normalmente, sólo se utilizará uno. En este caso, se ha usado este patrón para gestionar el movimiento de personajes. Un personaje puede moverse de forma aleatoria, en círculos, girar sobre sí mismo… Cada uno de estos movimientos se corresponde con una estrategia de movimiento diferente. El patrón permitirá, por una parte, agregar nuevos algoritmos de una forma sencilla y, por otra, contener el comportamiento de cada uno de forma independiente.

En este incremento se han usado también los patrones Helper (InitMenuController, GameController, WorldController, MapConstructor, Talker, SpanishTalker y DialogParser) y Singleton (StructDict, DecorationDict, ObjectDict, CharacterDict, AppearenceDict, FloorDict y MapDict).

6.1.6. DIAGRAMAS DE INTERACCIÓN

En este apartado se muestran los diagramas de secuencia creados para modelar las colaboraciones entre clases en la realización de las principales funcionalidades de este módulo. A mayores de los comentarios ya realizados de estos diagramas en el apartado 5.1.6, se aclararán ciertos aspectos por necesidad.

Siguiendo las recomendaciones de [FS99], sólo se mostrarán los valores de retorno que aporten claridad al diagrama. Por ejemplo, en la utilización de getters no se mostrará valor de retorno, pues se da por supuesto que el valor de retorno va implícito en el uso de la función.

Cumple destacar que el mejor diagrama de interacción no es el más complejo, ni el que incorpora todas las posibilidades, sino el que encapsula las principales funcionalidades y permite una sencilla interpretación de cómo se relacionan los diferentes objetos para satisfacer una determinada necesidad. En el caso de querer mostrar una variante de una funcionalidad o caso de uso principal, tal y como recomienda [FS99], se crearán diferentes diagramas. Por ejemplo, en el caso de uso UC-WS-01 *Mover al protagonista*, el usuario puede pedir mover al protagonista a una celda libre, a una celda ocupada o a una celda con entrada a otro mapa. En vez de mostrar todo este comportamiento en un mismo caso de uso, aumentando la complejidad, es recomendable utilizar tres diagramas diferentes y, de esta forma, la persona interesada en dicha funcionalidad puede escoger ver uno, dos o los tres diagramas en función de su interés.

Mover personajes

Este diagrama de secuencia (*véase* Figura 6.8) muestra las clases que colaboran para satisfacer FRQ-10 *Calcular movimiento de personajes*.

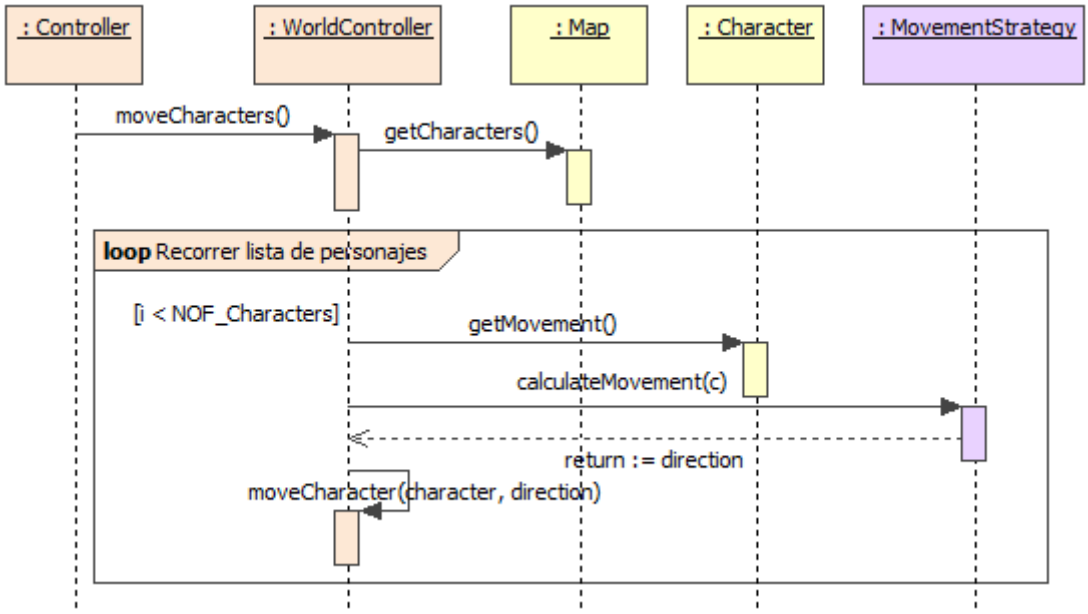

Figura 6.8. Diagrama de secuencia de FRQ-10 *Calcular movimiento de personajes*

Por cada iteración del juego, independientemente de las acciones del usuario, el controlador debe calcular el movimiento de los personajes en el mapa, exceptuando al protagonista. Esta responsabilidad la delega en WorldController, el cual obtiene del mapa actual la lista de personajes y el número de personajes que contiene. Por cada uno de ellos, aunque no se refleje en el diagrama, comprueba si el personaje está hablando ya que si aún lo está haciendo con el protagonista, no debería moverse. Si el personaje no está hablando se obtiene su tipo de movimiento y, en función del mismo, se escoge la estrategia de movimiento que corresponda. Se debe tener en cuenta que, si es un personaje sin movimiento, se pasará directamente al siguiente personaje. La estrategia devolverá la dirección calculada a la que el personaje se debe mover y esta se le envía a la función moveCharacter() junto al personaje para realizar el movimiento.

Mover protagonista

En la Figura 6.9 se muestra cómo colaboran las clases entre sí para completar la funcionalidad de mover personajes. El comportamiento es el mismo para mover al protagonista que a otro personaje cualquiera, ya que ambos son personajes. La única diferencia es que, para mover al protagonista, es necesaria una orden del usuario, mientras que el encargado de mover a cada uno de los personajes es WorldController.

En este escenario se contempla una situación en la que el personaje se mueve a una celda cuyo estado permite el movimiento y donde no hay otro personaje ni un objeto olvidado.

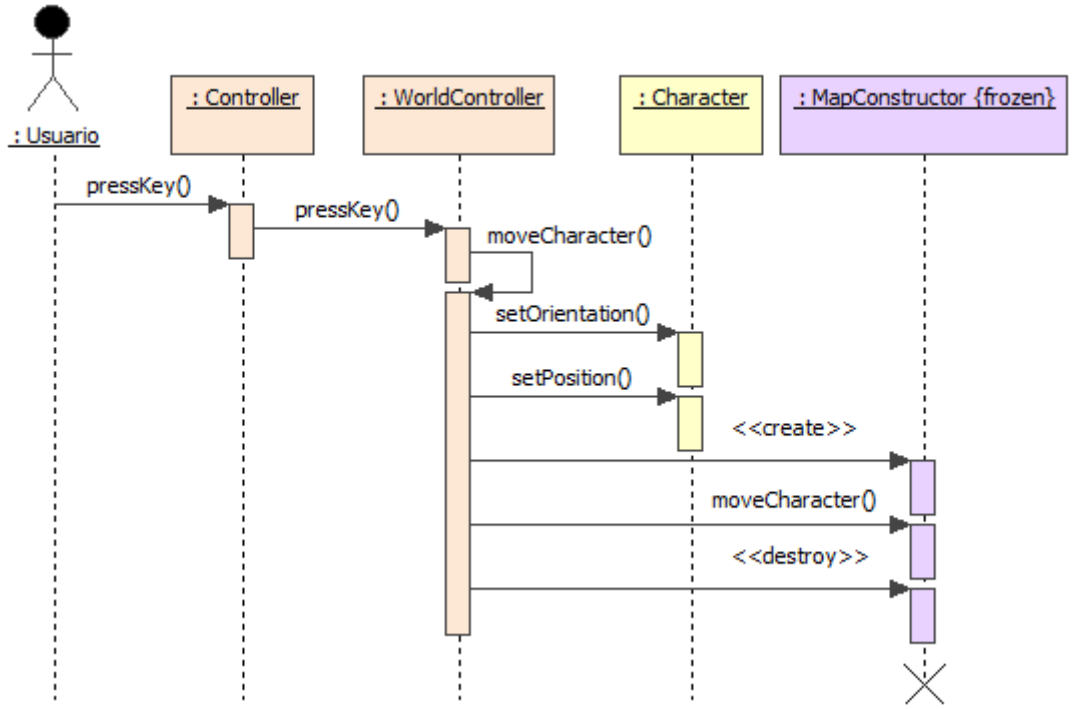

Figura 6.9. Diagrama de secuencia de UC-WS-01 *Mover al protagonista*

El controlador recibe la tecla pulsada y delega la tarea en WorldController. Éste, reconoce la tecla de movimiento y ejecuta moveCharacter() con el personaje (en este caso el protagonista) y la dirección correspondiente a la tecla de movimiento recibida. Lo primero que hace es actualizar la orientación y la posición del personaje, ya que esto se hace siempre independientemente de si el personaje se va a desplazar a otra celda. Una vez se han modificado los atributos, se comprueba, en función del estado de la celda solicitada y de si existe un objeto olvidado u otro personaje en dicha celda, si se puede realizar el desplazamiento. Si el movimiento se puede realizar, se crea el constructor de mapas y se solicita la operación moveCharacter() que actualizará la localización del personaje.

Mover al protagonista: Entrada a otro mapa

En uno de los diferentes escenarios alternativos de mover al protagonista, el jugador solicita el acceso a una celda con entrada a otro mapa.

Tal y como se muestra en el diagrama de secuencia de la Figura 6.10, el comportamiento es idéntico al anterior hasta la auto-llamada a la función moveCharacter(). En este caso, WorldController detecta que la celda solicitada contiene una entrada. Una vez detectado este caso, se obtiene la entrada para conocer el mapa al que debe cambiar y la posición del protagonista en el nuevo mapa. A partir del identificador de mapa que contiene dicha entrada, se carga el mapa correspondiente a través de la función getMap() de iMapDict. A continuación, se actualiza el mapa para realizar los cambios correspondientes a los eventos realizados por el jugador. Una vez el mapa está definitivamente preparado, se crea la instancia de MapConstructor y se añade el jugador al mapa con la nueva posición.

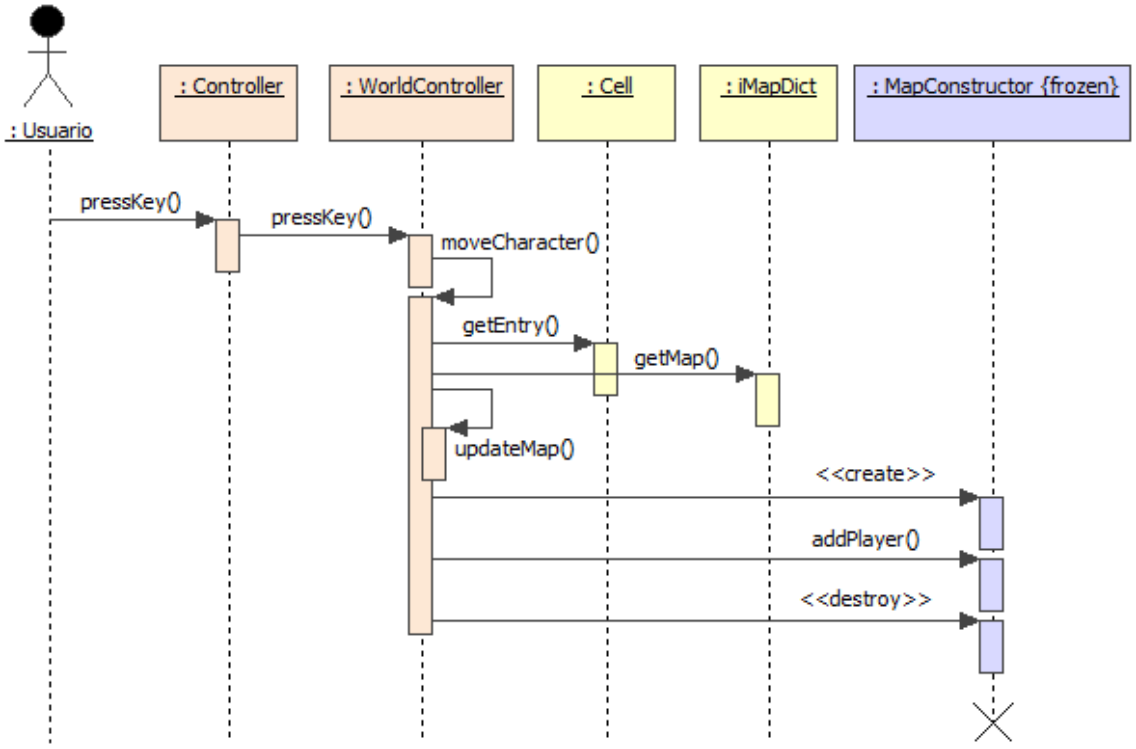

Figura 6.10. Diagrama de secuencia de UC-WS-02 *Acceder a una entrada*

Dibujar menú inicial

Uno de los diagramas que más claridad aporta al funcionamiento del sistema es el dibujar el contenido de la pantalla. En la Figura 6.11 se muestra el diagrama de secuencia para la funcionalidad de dibujar la pantalla, en este caso, la interfaz de menú inicial. En este diagrama se observa cómo las nuevas clases se relacionan con las ya implementadas en el módulo anterior.

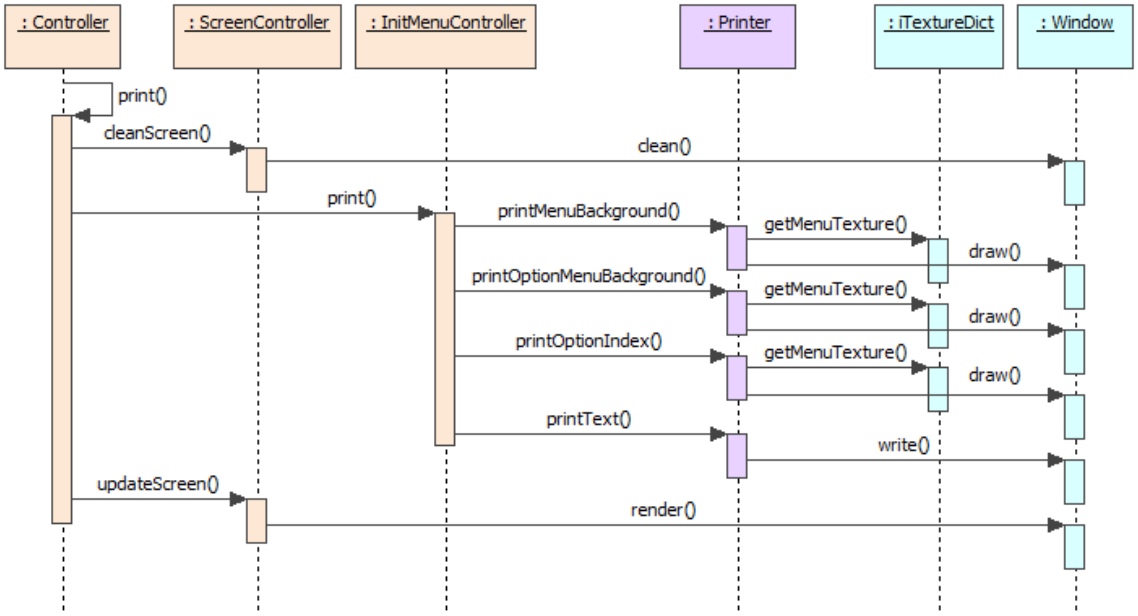

Figura 6.11. Diagrama de secuencia de dibujado del menú inicial

El diagrama comienza con la auto-llamada de dibujar de Controller. Lo primero que hace la función print() es limpiar la pantalla borrando su contenido. Una vez la pantalla está vacía (color negro), delega en InitMenuController la responsabilidad de dibujar el contenido de la interfaz. Esta clase, que contiene la lógica de presentación, le indica a Printer qué debe dibujar y dónde. Printer obtiene las texturas del diccionario a partir de los datos proporcionados por el controlador y le indica a la ventana qué textura debe dibujar y en dónde. Una vez la ventana se ha terminado de dibujar, la ventana se actualiza conforme el nuevo contenido por orden de ScreenController.

Dibujar modo mundo

A continuación se muestra en la Figura 6.12 el diagrama de secuencia que contiene el comportamiento que permite dibujar la interfaz modo mundo.

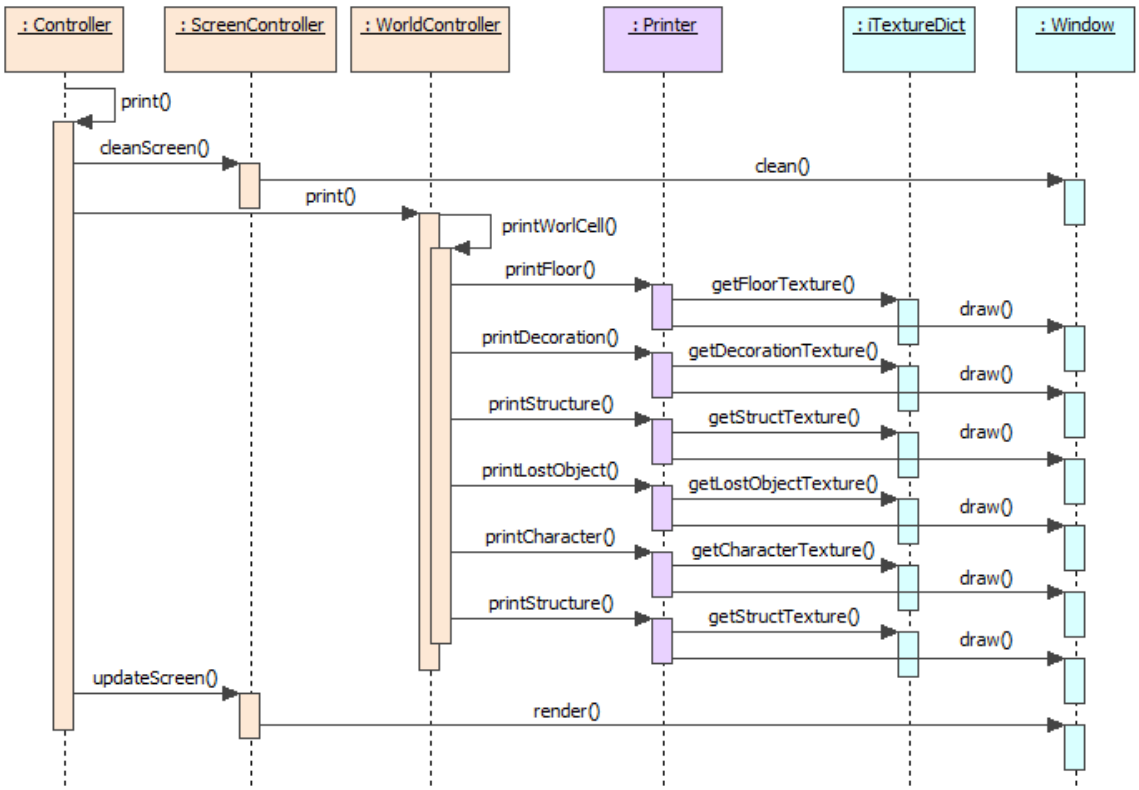

Figura 6.12. Diagrama de secuencia de dibujado de una celda del modo mundo

El comportamiento es idéntico a Figura 6.11. Diagrama de secuencia de dibujado del menú inicial a excepción de la lógica de presentación. Esto es el resultado de un correcto desacoplamiento de responsabilidades, permitiendo que cada controlador contenga la forma y contenidos a dibujar y que, las operaciones comunes, sean las mismas para todas las interfaces.

Este diagrama refleja cómo se realiza el dibujado de una celda. En la realidad, la llamada a print() de WorldController se encargará de recorrer todas las celdas visibles del mapa y dibujar cada una de ellas. Tal y como se ha explicado en la fase de análisis general, a la hora de dibujar un mapa se deben dibujar todas las capas una por una. Para cada una de las capas, WorldController le dice a Printer el contenido, en caso de existir, que debe dibujar. Printer obtiene, a partir de los datos recibidos, la textura y le envía a la ventana la referencia de la textura a dibujar y la posición de pantalla donde debe hacerlo.

Hablar con un personaje

La Figura 6.13 muestra el diagrama de secuencia para el escenario principal de UC-WS-05 *Dialogar con un personaje*.

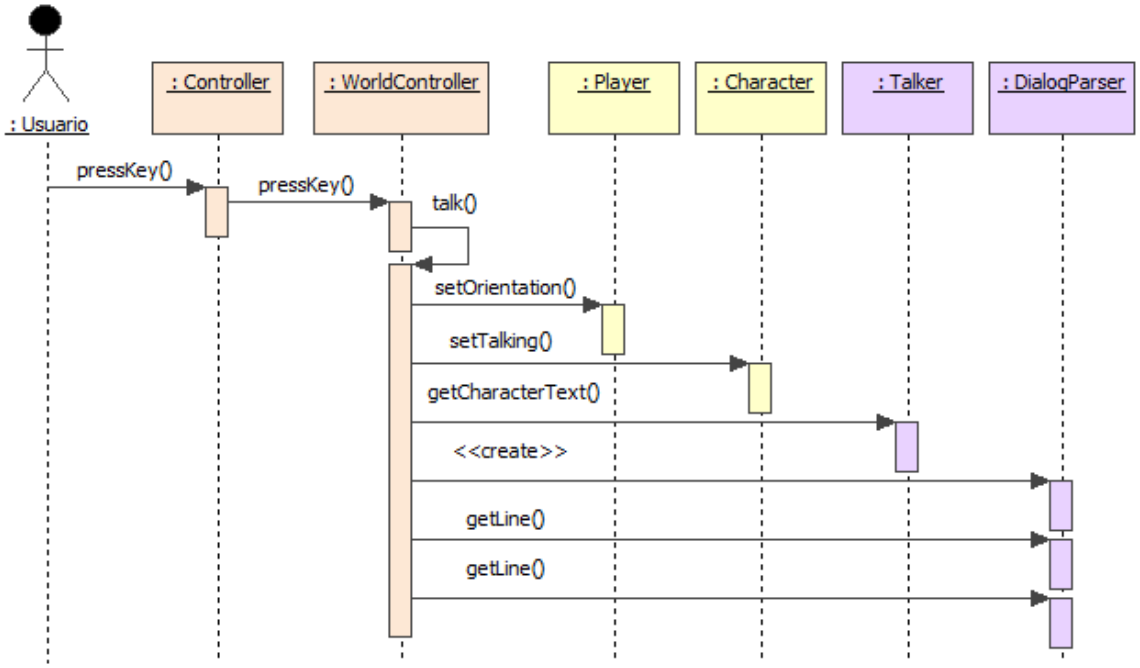

Figura 6.13. Diagrama de secuencia de UC-WS-05 *Dialogar con un personaje*

El controlador recoge la tecla de Aceptar pulsada por el usuario y delega el control en WorldController. Éste, reconoce la tecla y llama a la función talk(). Lo primero que hace la función es comprobar que exista un personaje en la celda solicitada y cambia su orientación y posición para que "mire" al protagonista. Obtiene el texto del personaje en el idioma activo a través de Talker. Posteriormente, obtiene las dos primeras líneas de diálogo para poder mostrar el texto con la longitud adecuada por pantalla.

Cargar partida

La Figura 6.14 muestra el diagrama para el escenario principal de UC-GS-02 *Continuar partida*.

El usuario, situado en la opción "Continuar partida" del menú inicial pulsa la tecla Aceptar. El controlador principal delega en InitMenuController la responsabilidad de interpretar la tecla pulsada. Éste ejecuta loadGame() y le pide a GameController que cargue los datos de la partida desde fichero. Una vez los datos están cargados, el controlador cambia la vista a *modo mundo* y le pide a WorldController que inicialice los datos de acuerdo al contenido de la partida. Para ello, obtiene el mapa adecuado desde iMapDict, lo actualiza con los valores de los eventos de historia y crea y añade el protagonista al mapa actual.

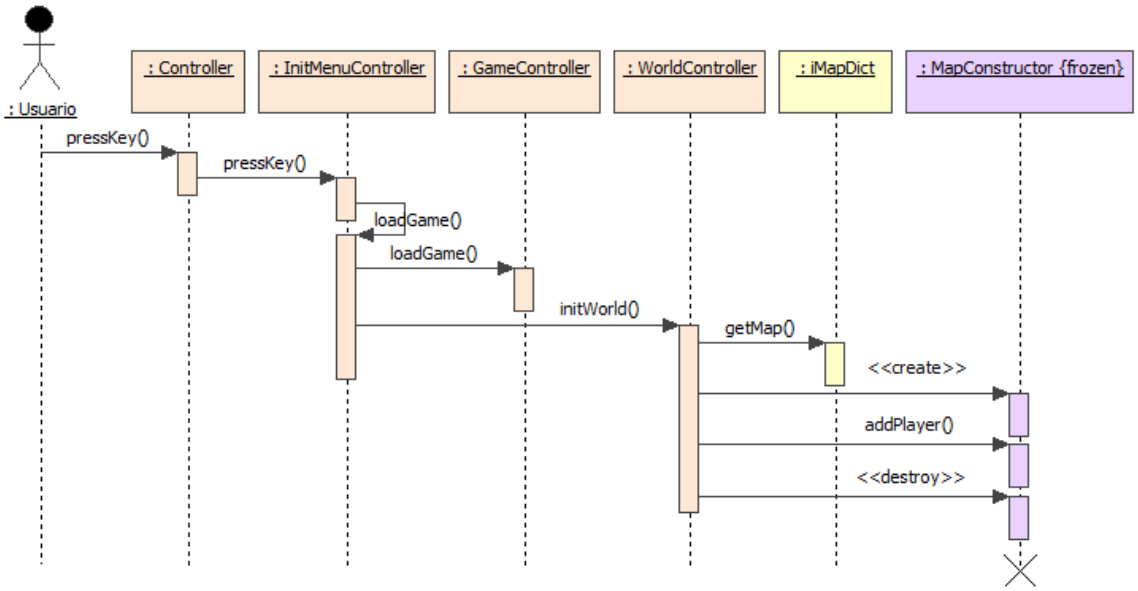

Figura 6.14. Diagrama de secuencia de UC-GS-02 *Continuar partida*

6.1.7. Diagramas de actividad

Los diagramas de actividad son muy útiles para mostrar y, sobre todo, promover el comportamiento en paralelo. Estos diagramas muestran diferentes actividades que se relacionan entre sí a través de disparadores para así conformar el flujo de trabajo de una determinada funcionalidad. Su principal virtud, como ya se ha mencionado, es la de evitar procesos secuenciales innecesarios y descubrir oportunidades para paralelizar procesos, mejorando así la eficiencia y capacidad de respuesta de los procesos del negocio. También permiten visualizar cuales son los hilos y cuándo necesitan sincronizarse.

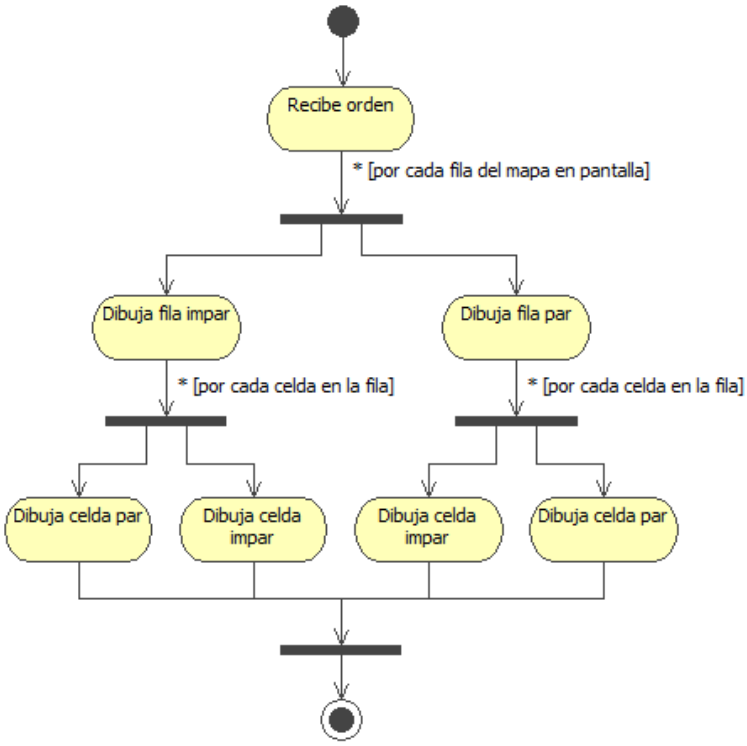

Figura 6.15. Diagrama de actividad de dibujado del mapa

En el diagrama de la Figura 6.15 se muestra el diagrama de actividad para la funcionalidad encargada de dibujar el mapa en pantalla. El proceso de dibujar es, sin duda, uno de los procesos más complejos y que requieren mayor tiempo de procesamiento. Por lo tanto, se ha intentado optimizar paralelizando actividades de acuerdo a NFR-PRQ-04 *Velocidad de operaciones complejas*. Es cierto que el proceso de dibujar el contenido de una celda no se puede paralelizar ya que es importante mantener el orden a la hora de dibujarla, pero sí se puede hacer a la hora de recorrer el mapa para su dibujado. Existirán cuatro hilos encargados de dibujar las filas y columnas pares e impares del mapa reduciendo notablemente el tiempo requerido para completar el proceso. Cabe destacar, que esta decisión será especialmente optima debido a la previa implementación del diccionario de datos de texturas como patrón Singleton.

6.2. Detalles de implementación

En este apartado se justificarán las decisiones tomadas a la hora de implementar las funcionalidades diseñadas para éste módulo.

Diccionario de mapas

Los mapas son, junto a las texturas, los elementos de la aplicación de mayor tamaño pero, a diferencia de estas, las veces que son llamados son mucho menores. Por estos motivos, se ha diseñado e implementado el diccionario de mapas como un diccionario que no almacena los mapas sino que, con cada petición, los carga y los devuelve. Cada mapa está contenido en un archivo diferente, facilitando así su identificación y edición. Estos archivos contienen una única función: almacenar la información del mapa. A continuación se muestra un pequeño ejemplo:

```
Map* getTestMap(){

        //  Creamos el mapa y asignamos el identificador y el tamaño
        Map* m = new Map(mTestRoute01, 50, 50);

        //  Asignamos el nombre al mapa
        m->setName("Test map");

        //  Creamos el constructor del mapa
        MapConstructor* mc = new MapConstructor(m);

        //  Creamos el suelo
        mc->addFloor(fGreenGrass01);

        //  Creamos los personajes
        mc->addCharacter(TestMap01_BugCatcher01);
        mc->addCharacter(TestMap01_BugCatcher02);
        mc->addCharacter(TestMap01_Jogger01);
        mc->addCharacter(TestMap01_BugCatcher03);
        mc->addCharacter(TestMap01_MaleCyclist01);
        mc->addCharacter(TestMap01_FemaleCyclist01);

        //  Creamos los objetos perdidos
        mc->addLostObject(Object::PokeBall, 2, 2, TestMap01_PokeBall01);
        mc->addLostObject(Object::SuperPotion, 12, 7, TestMap01_SuperPotion01);
        mc->addLostObject(Object::MaxPotion, 16, 19, TestMap01_MaxPotion01);

        //  Creamos las estructuras
        mc->addStruct(CrackedRock01, 3, 3);
        mc->addStruct(CrackedRock01, 3, 4);
```

```
	mc->addStruct(MovableRock01, 3, 5);
	mc->addStruct(Tree01, 12, 6);
	mc->addStruct(CuttableTree01, 13, 9);

	//  Creamos las entradas
	mc->addEntry(mTestRoute02, 16, 22, 14, 38);

	//  Creamos los eventos pasivos
	mc->addCharacterRemovalEvent(TalkBugCatcher, TestMap01_BugCatcher03);
	mc->addCharacterCreationEvent(TalkBugCatcher, TestMap01_BugCatcher04);
	mc->addCharacterRemovalEvent(TalkBugCatcher, TestMap01_FemaleCyclist01);
	mc->addCharacterRemovalEvent(TalkBugCatcher, TestMap01_MaleCyclist01);

	//  Eliminamos el creador de mapas
	delete mc;

	//  Devolvemos el mapa
	return m;
}
```

Gestión de los idiomas

Aunque de momento sólo exista un idioma para la aplicación, se ha considerado necesario en los requisitos la necesidad de construir el sistema de tal forma que la adición de nuevos lenguajes sea lo más sencilla posible en el futuro. Para conseguirlo, se ha almacenado en la clase SpanishTalker todos los textos de la aplicación. Cuando se quiera añadir un nuevo idioma bastará con duplicar la clase y traducir cada uno de estos textos.

La clase Talker es además la responsable de garantizar que se realiza correctamente la asignación de memoria a cada uno de los textos. Para ello, se utiliza el siguiente método:

```
char* SpanishTalker::allocateText(char* t){

	//  Se contempla el carácter de fin de cadena para calcular el tamaño
	int size = strlen(t) + 1;

	char* text = new char[size];
	strncpy_s(text, size, t, size);

	//  Es responsabilidad de la clase invocadora liberar la memoria reservada
	return text;
}
```

Cada una de las funciones que contienen los diferentes textos hacen uso de esta función:

```
char* SpanishTalker::getStructText(int id){
	switch (id){
	case CrackedRock01:
		return allocateText("Has roto la roca que no te dejaba pasar");
	case CuttableTree01:
		return allocateText("Has cortado el árbol que te impedía el paso");
	case MovableRock01:
		return allocateText("Creo que puedo mover esta roca...");
	default:
		return NULL;
	}
}
```

Programación multihilo con OpenMP

En el apartado 6.1.7 se ha comentado la necesidad de paralelizar ciertas partes del código para mejorar la eficiencia de la aplicación. Para ello, se utiliza la API de programación multiproceso OpenMP, que permite añadir concurrencia al sistema en base al modelo de ejecución fork-join[8]. Esta API está compuesta por un conjunto de directivas de compilador, rutinas de biblioteca y variables de entorno que modifican el comportamiento en tiempo de ejecución.

El diagrama de actividad de la Figura 6.15 muestra un ejemplo de cómo paralelizar a la hora de dibujar el modo mundo. En dicho ejemplo se propone la creación de cuatro hilos, los cuales, trabajando conjuntamente, dibujan dicha interfaz de forma más rápida. Sin embargo, debe entenderse que dicho diagrama refleja la posibilidad de incorporar comportamiento paralelo. No todos los sistemas sobre los que va a funcionar la aplicación tendrán cuatro núcleos y algunos tendrán incluso más. Teniendo esto presente se puede aprovechar las virtudes que OpenMP proporciona al programador para repartir la tarea de dibujar en tantos hilos como se pueda ya que, por defecto, al crear un área de procesamiento en paralelo, OpenMP crea tantos hilos como núcleos haya para así optimizar el rendimiento. Se puede saber cuántos hilos se crean con la función:

```
int omp_get_num_threads();
```

Para definir el código a ejecutar por los distintos hilos se usa la siguiente directiva:

```
#pragma omp parallel shared(x,y) private(i)
```

Las opciones “shared” y “private” indican las variables compartidas (*x* e *y*) y privadas (*i*) para cada uno de los hilos. El hilo principal, es decir, el proceso principal es el 0, es el que existe antes de iniciar la región y el que continúa cuando esta termina.

Para paralelizar un bucle *for*, es decir, repartir X iteraciones del bucle entre cada uno de los hilos, OpenMP proporciona la directiva:

```
#pragma omp for nowait reduction(+:r)
```

Ambas directivas son combinables, de hecho, es recomendable hacerlo siempre que sea posible. Esta segunda directiva reparte iteraciones del bucle entre el grupo de hilos. Contiene además, un comando extra: *Schedule*; el cual permite, entre otras cosas, indicar el chunk de operaciones, es decir, el número de instrucciones que ejecutará cada hilo. En su ausencia, el propio procesador calcula el chunk de instrucciones por hilo, dividiendo el número total de iteraciones entre el número de hilos. Se ha comprobado que éste método es más eficiente.

Ya que el dibujado de una celda es independiente del de otra, se ha usado la directiva *nowait*, la cual indica que los hilos ejecutan bloques de instrucciones de forma continuada, sin esperar a que los otros hilos hayan terminado su bloque. La directiva *reduction* indica que se hace una copia privada de la variable r por cada hilo y el operador ‘+’ implica que, una vez finalizados todos los hilos, se sumará el contenido cada una de estas copias.

[8] Fork-join es un paradigma que proviene de los sistemas Unix, donde una cierta tarea muy pesada se divide en K hilos (fork) con menor peso, para luego "recolectar" sus resultados al final y unirlos en un solo resultado (join).

7. Desarrollo: Tercera iteración

Ya se adelantó en el apartado 4.2.1 que en esta iteración se desarrollarán las funcionalidades relacionadas con los combates entre Pokémon. Esto incluye el almacenamiento de toda la información necesaria y la lógica de combate. Terminado este incremento, se dispondrá del software completo, a la falta del desarrollo de alguna funcionalidad opcional, donde el usuario podrá luchar contra los diferentes Pokémon salvajes con todas las características que eso implica: perder vida, subir de nivel, evolucionar, huir del combate…

Al igual que en fases anteriores, se comenzará esta sección revisando los requisitos identificados y los riesgos del proyecto. Nuevamente, no se realizarán modificaciones. A continuación, se mostrarán los aspectos de diseño e implementación y, finalmente, las pruebas del módulo construido.

7.1. Diseño

En este apartado se mostrarán las diferentes clases diseñadas, agrupadas en los diferentes paquetes, tanto en los ya existentes Controllers y Helpers como en los nuevos Pokémon y LevelingRateStrategy. Para describir la estructura del sistema, se utilizarán diagramas de clases y, para mostrar las dependencias entre los diferentes paquetes, un diagrama de paquetes.

7.1.1. Diagrama de paquetes

Como se puede observar en el diagrama de la Figura 7.1, el paquete Controller aumenta su contenido con un nuevo sub-paquete: LevelingRateStrategy. Este paquete contiene todos los posibles tipos de crecimiento de los Pokémon y proporciona una clase abstracta de acceso a cada uno de estos elementos.

El modelo también ve aumentado su volumen con el nuevo sub-paquete Pokémon, que dispone de todas las clases del modelo relacionados con estas criaturas.

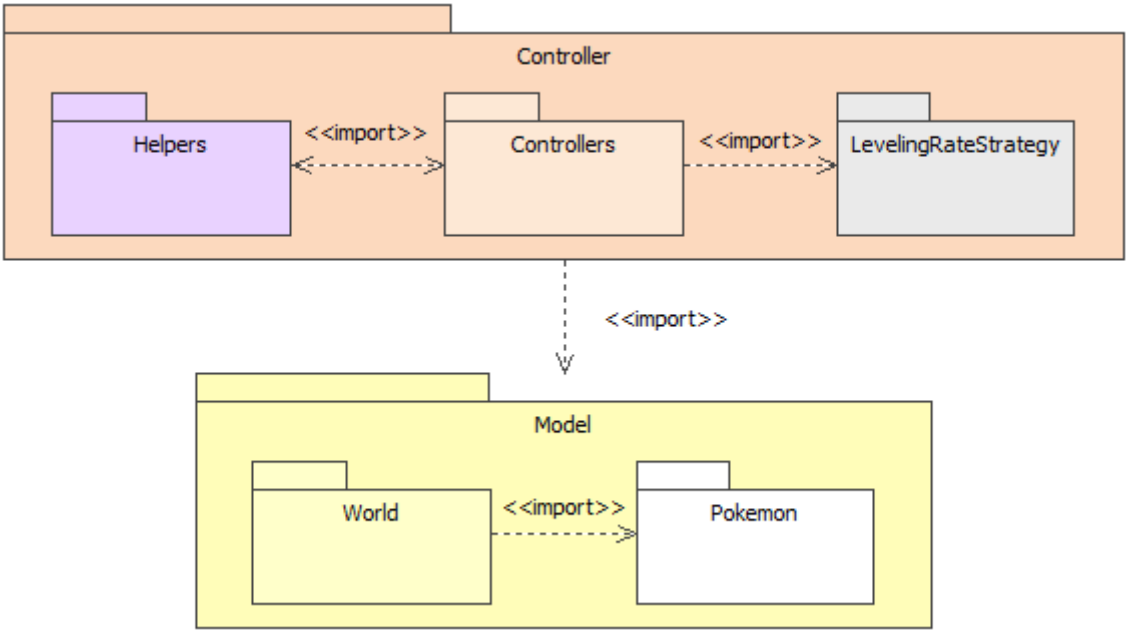

Figura 7.1. Diagrama de paquetes – Tercera iteración

A su vez, los paquetes Controllers y Helpers, ven aumentada su complejidad por la incorporación de nuevas clases. World, por otra parte, mantiene todas las clases y sólo se muestra en el diagrama para ilustrar la relación que guarda con el paquete Pokémon.

7.1.2. Paquete Controller

Paquete Controllers

En primer lugar se presenta, en la Figura 7.2, el diagrama de clases para el paquete Controllers. Sólo se muestran las operaciones y atributos de la clase añadida BattleController y de Controller. El resto de clases, mantienen sus atributos y operaciones.

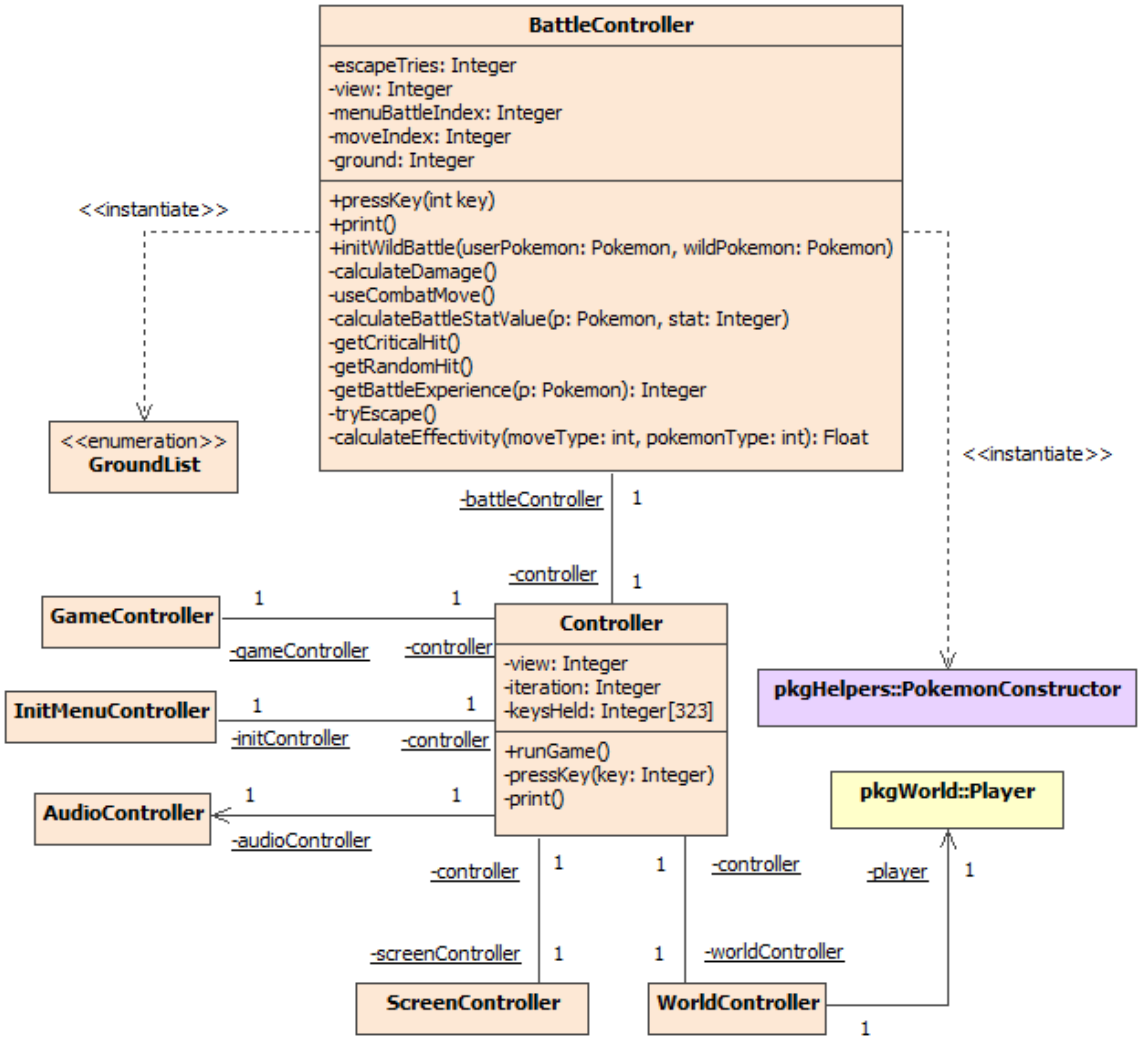

Figura 7.2. Diagrama de clases para el paquete Controllers

A continuación, utilizando la plantilla contenida en la Tabla 5.1, se presenta la tarjeta CRC generada para la nueva clase BattleController. La enumeración GroundList mantiene la misma responsabilidad que las enumeraciones descritas con anterioridad, esto es, proveer identificadores a los objetos que dependen de ellas.

BattleController	
Responsabilidad	**Colaboración**
Gestionar las funcionalidades de combate	PokemonConstructor
Contener la lógica de presentación de la interfaz *modo combate*	Printer

Tabla 7.1. Tarjeta CRC de la clase BattleController

Paquete Helpers

En la Figura 7.3, al igual que ocurría anteriormente, se muestra las relaciones entre las diferentes clases que componen el paquete Helpers y sólo se especifican las nuevas operaciones y atributos de la nueva clase PokemonConstructor.

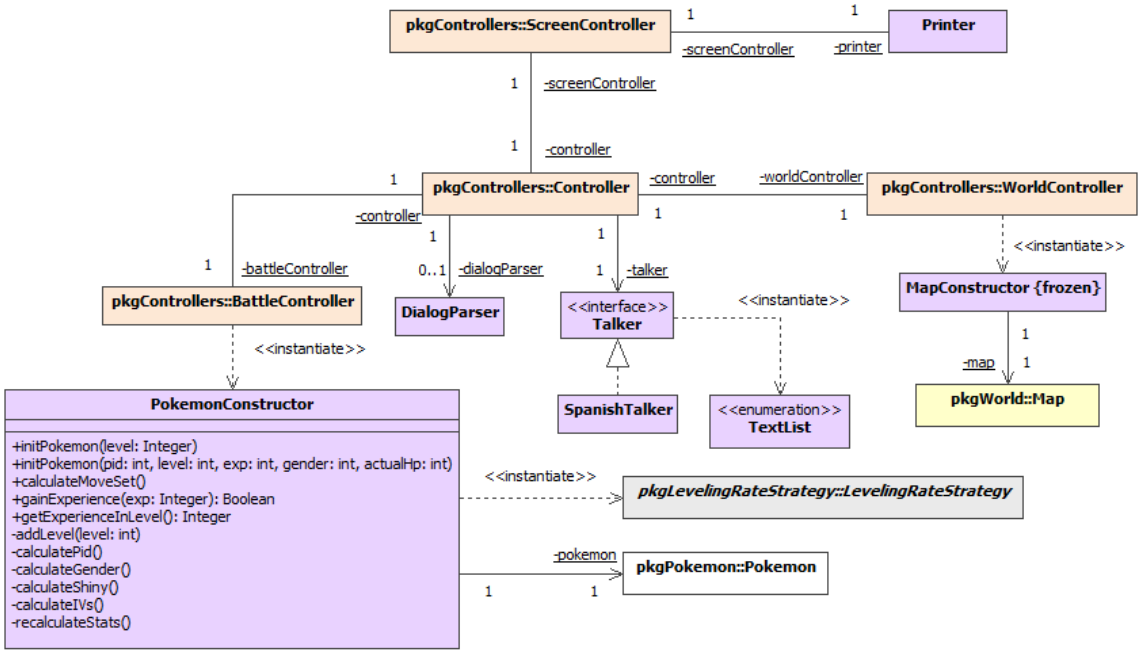

Figura 7.3. Diagrama de clases para el paquete Helpers

A continuación se muestra la tarjeta CRC que refleja las responsabilidades de PokemonConstructor:

PokemonConstructor	
Responsabilidad	**Colaboración**
Gestionar las diferentes operaciones complejas que se realizan sobre los Pokémon	Pokemon LevelingRateStrategy

Tabla 7.2. Tarjeta CRC de la clase BattleController

Paquete LevelingRateStrategy

El diagrama de clases para el paquete LevelingRateStrategy, que se puede observar en la Figura 7.4, muestra las diferentes clases que gestionan el crecimiento de los Pokémon.

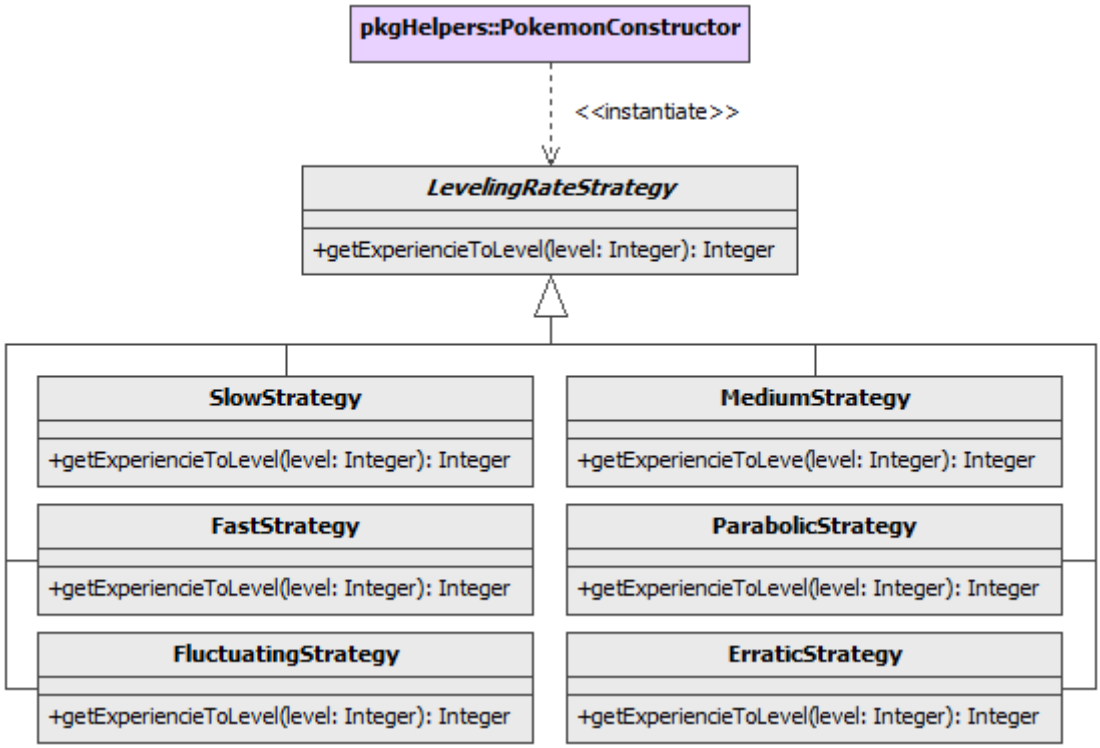

Figura 7.4. Diagrama de clases del paquete LevelingRateStrategy

A continuación, utilizando la plantilla contenida en la Tabla 5.1, se presentan las tarjetas CRC generadas para las clases del paquete.

LevelingRateStrategy	
Responsabilidad	**Colaboración**
Proporcionar un punto de acceso para el cálculo de cada uno de los tipos de crecimiento Pokémon	SlowStrategy MediumStrategy FastStrategy ParabolicStrategy FluctuatingStrategy ErraticStrategy

Tabla 7.3. Tarjeta CRC de la clase LevelingRateStrategy

SlowStrategy	
Responsabilidad	**Colaboración**
Calcular los puntos de experiencia necesarios para alcanzar cada nivel siguiendo una estrategia de crecimiento lento	

Tabla 7.4. Tarjeta CRC de la clase SlowStrategy

MediumStrategy	
Responsabilidad	**Colaboración**
Calcular los puntos de experiencia necesarios para alcanzar cada nivel siguiendo una estrategia de crecimiento medio	

Tabla 7.5. Tarjeta CRC de la clase MediumStrategy

FastStrategy	
Responsabilidad	**Colaboración**
Calcular los puntos de experiencia necesarios para alcanzar cada nivel siguiendo una estrategia de crecimiento rápido	

Tabla 7.6. Tarjeta CRC de la clase FastStrategy

ParabolicStrategy	
Responsabilidad	**Colaboración**
Calcular los puntos de experiencia necesarios para alcanzar cada nivel siguiendo una estrategia de crecimiento parabólico	

Tabla 7.7. Tarjeta CRC de la clase ParabolicStrategy

FluctuatingStrategy	
Responsabilidad	**Colaboración**
Calcular los puntos de experiencia necesarios para alcanzar cada nivel siguiendo una estrategia de crecimiento fluctuante	

Tabla 7.8. Tarjeta CRC de la clase FluctuatingStrategy

ErraticStrategy	
Responsabilidad	**Colaboración**
Calcular los puntos de experiencia necesarios para alcanzar cada nivel siguiendo una estrategia de crecimiento errático	

Tabla 7.9. Tarjeta CRC de la clase ErraticStrategy

7.1.3. Paquete Model

Paquete Pokemon

En la Figura 7.5 se muestra el diagrama de clases del paquete Pokemon. Igual que sucedía anteriormente con el paquete World, se han utilizado cuatro colores diferentes para facilitar la visión general del paquete. Cada color identifica a un “tipo” de clase: gris a las enumeraciones, verde a las interfaces de los diccionarios, azul a las implementaciones de los diccionarios y blanco al resto del modelo.

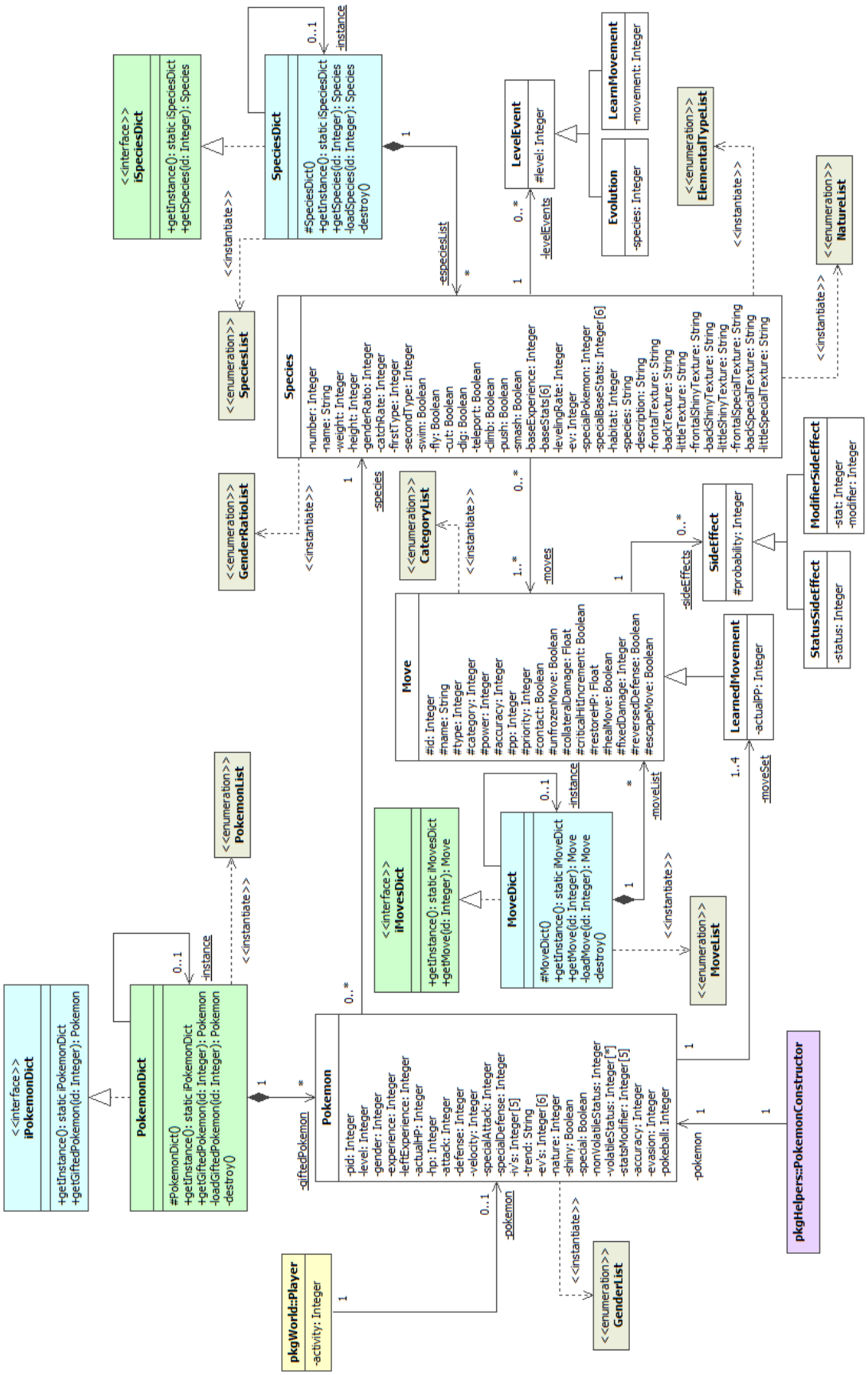

Figura 7.5. Diagrama de clases del paquete Pokemon

Debido a que todas las clases del mismo "tipo" tienen las mismas responsabilidades, se ha decidido hacer sólo una tarjeta CRC por cada uno de estos grupos de clases, entendiendo que el propio nombre de la clase y el diagrama son suficientes para comprender con qué tipo de datos trabaja para satisfacer su responsabilidad.

Enumeraciones	
Responsabilidad	**Colaboración**
Almacenar y proveer los identificadores de los diferentes componentes de los Pokémon	

Tabla 7.10. Tarjeta CRC común a todas las enumeraciones del paquete Pokemon

Interfaces de diccionario	
Responsabilidad	**Colaboración**
Proporcionar un punto de acceso para la obtención de cada uno de los elementos contenidos en los diccionarios	

Tabla 7.11. Tarjeta CRC común a todas las interfaces del paquete Pokemon

Diccionarios de datos	
Responsabilidad	**Colaboración**
Almacenar y proveer referencias a los elementos presentes en los Pokémon	

Tabla 7.12. Tarjeta CRC común a todos los diccionarios del paquete Pokemon

Clases de Modelo	
Responsabilidad	**Colaboración**
Contener la estructura de los Pokémon y sus componentes	

Tabla 7.13. Tarjeta CRC común a todas las clases de Modelo del paquete Pokemon

7.1.4. Patrones de diseño

En este incremento no se han utilizado nuevos patrones de diseño por lo que no se explicarán de nuevo cada uno de ellos sino que se indicará dónde se han utilizado.

Ya que existen diferentes formas en las que una especie Pokémon aumenta de nivel, se ha optado por incorporar dichas estrategias con un patrón Strategy. Esto permitirá escoger dinámicamente el algoritmo a utilizar para calcular cualquier valor relacionado con el nivel y la experiencia en función del tipo de crecimiento de la especie del Pokémon.

Nuevamente, los diccionarios han sido implementados utilizando el patrón Singleton y se han añadido los Helpers BattleController y PokemonConstructor.

7.1.5. DIAGRAMAS DE INTERACCIÓN

En este apartado se muestran los diagramas de secuencia creados para modelar las colaboraciones entre clases en la realización de las principales funcionalidades de este módulo. Las explicaciones pertinentes a este tipo de diagramas pueden revisarse en los apartados 5.1.6 y 6.1.6.

La mayor parte de las funcionalidades diseñadas son similares a funciones de anteriores iteraciones y, por ello, no se profundizará demasiado en este apartado.

Dibujar combate

El diagrama de secuencia de la Figura 7.6 muestra la colaboración entre clases que satisfacen la funcionalidad de dibujar la interfaz modo batalla, en un escenario donde el sistema informa al usuario, por ejemplo, del movimiento que ha usado el Pokémon salvaje.

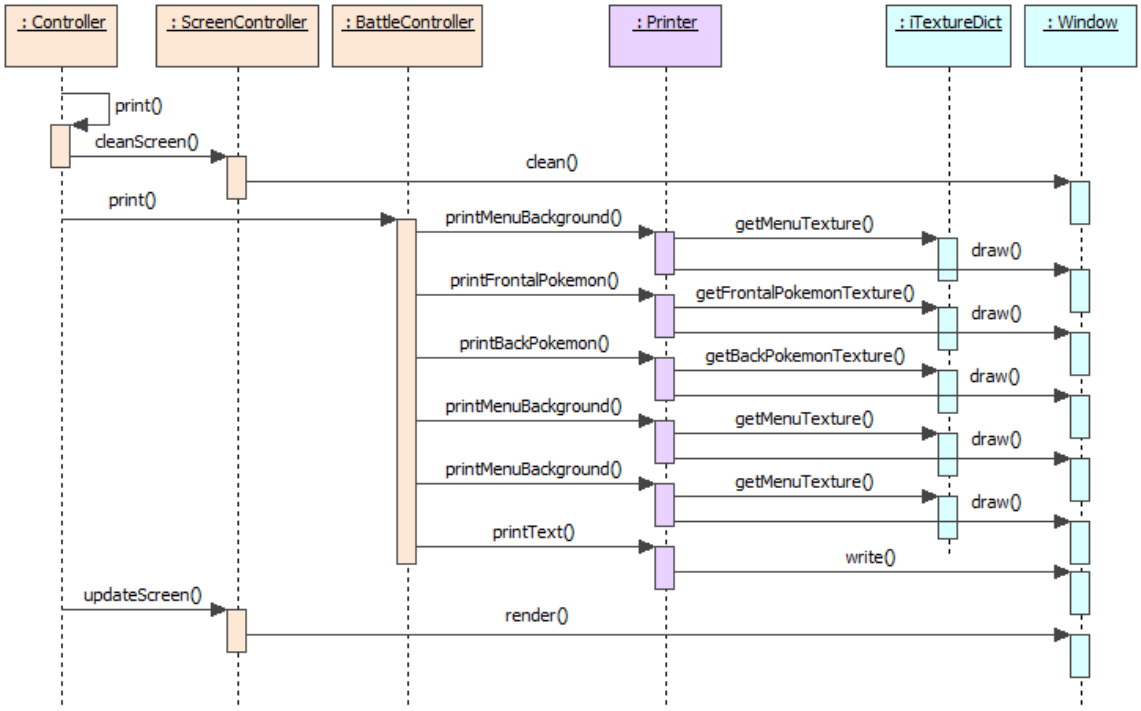

Figura 7.6. Diagrama de secuencia para la funcionalidad de dibujar combate

Al igual que en otros diagramas de dibujo de interfaz, en este también se vacía el contenido de la pantalla en primer lugar. A continuación, se dibuja el fondo de la batalla en función del tipo de suelo en el que se desarrolla la batalla, la parte delantera del Pokémon atacante y la trasera del defensor, sus barras de vida y, finalmente, se dibuja el texto que hay guardado. En último lugar, se actualiza la pantalla.

Próximamente, en el apartado 7.1.6, se muestra cómo esta operación puede ser paralelizada para obtener mejor rendimiento.

Encontrar Pokémon salvaje

En la Figura 7.7 se muestra mediante un diagrama de secuencia la colaboración entre clases para satisfacer FRQ-16 *Calcular encuentro Pokémon*. Este diagrama refleja un escenario en el que el protagonista accede a una celda de captura y el cálculo aleatorio implica la aparición de un Pokémon salvaje y, por tanto, el comienzo de un combate.

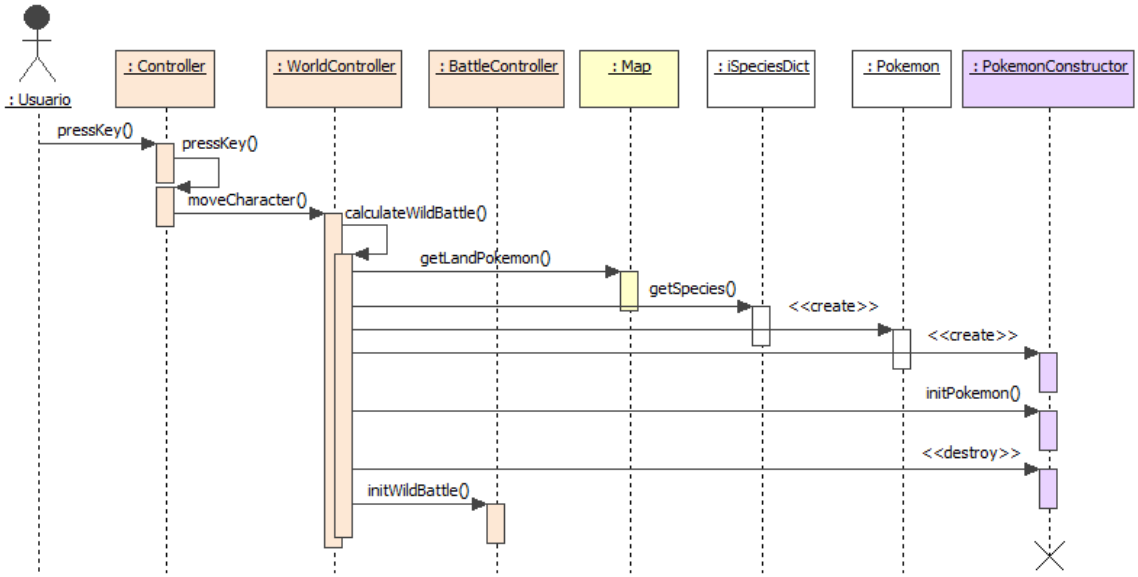

Figura 7.7. Diagrama de secuencia para FRQ-16 *Calcular encuentro Pokémon*

Una vez WorldController ha realizado el movimiento solicitado por el usuario, detecta que el protagonista se encuentra en una celda de captura y realiza el cálculo para saber si aparece un Pokémon salvaje. Dado que este escenario contempla que el Pokémon aparece, WorldController obtiene la lista de Pokémon salvajes que pueden aparecer en el mapa en el terreno en el que el jugador se encuentra, en este caso en tierra. Aleatoriamente, y en función del ratio de aparición de cada especie, se obtiene la especie de Pokémon salvaje en el diccionario. WorldController crea la instancia del nuevo Pokémon y mediante el constructor de Pokémon inicializa sus valores mediante los cálculos de IV's, PID, movimientos, género y si es o no un Pokémon shiny. Finalmente, WorldController inicializa la batalla en BattleController y se cambia la vista al nuevo tipo de interfaz.

Subir de nivel

El diagrama de secuencia de la Figura 7.8 muestra la colaboración entre clases que satisfacen el requisito funcional FRQ-15 *Subir de nivel*. En este escenario, un Pokémon del usuario derrota a su rival y obtiene suficientes puntos de experiencia como para subir de nivel.

En primer lugar, se añade la experiencia ganada al Pokémon. Una vez se dispone de la experiencia total, se comprueba si la experiencia ganada es superior a la experiencia que restaba por ganar. En dicho caso, se debe determinar la experiencia que hace falta para subir al próximo nivel mediante la interfaz LevelingRateStrategy. Una vez obtenidos los puntos de experiencia necesarios, se actualiza el nivel y la experiencia restante, y se recalculan las estadísticas totales del Pokémon. Finalmente, se corrigen los HP actuales del Pokémon de tal forma que se mantenga el porcentaje de puntos de vida. De lo contrario, un Pokémon con el 100% de vida, no mantendría la máxima salud al subir de nivel.

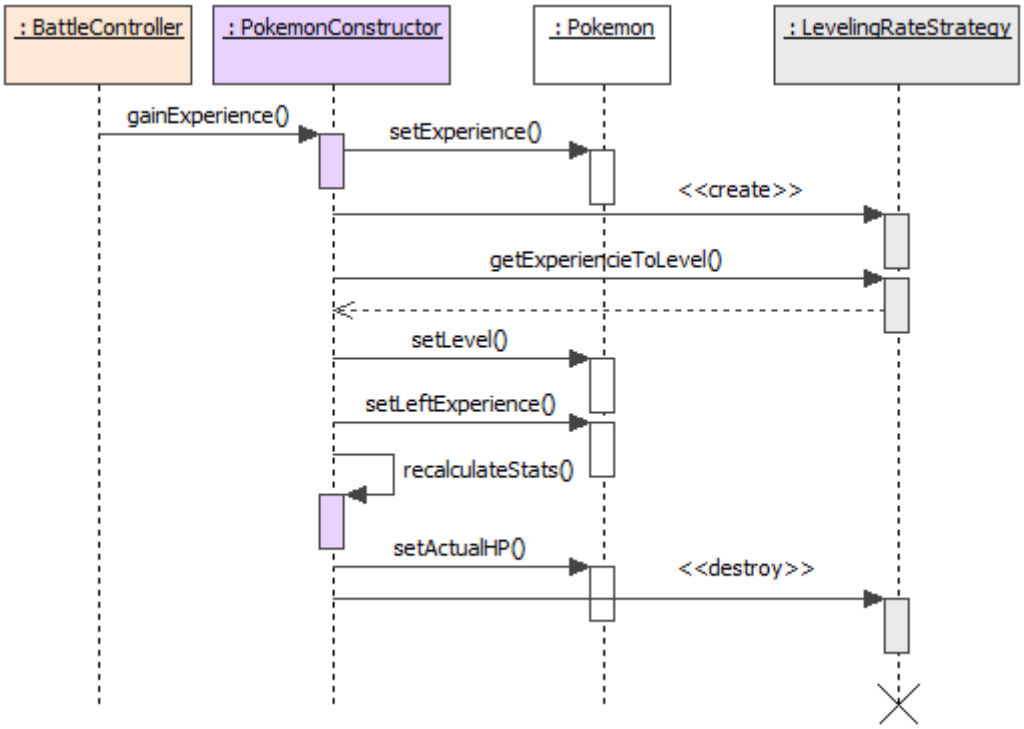

Figura 7.8. Diagrama de secuencia para FRQ-15 *Subir de nivel*

7.1.6. Diagramas de actividad

Al igual que sucedía en el apartado 6.1.7, el dibujado de la batalla es una de las funciones que más procesamiento requieren y que más ralentizan el sistema. Por ello, es recomendable paralelizarlo, tal y como se muestra en la Figura 7.9.

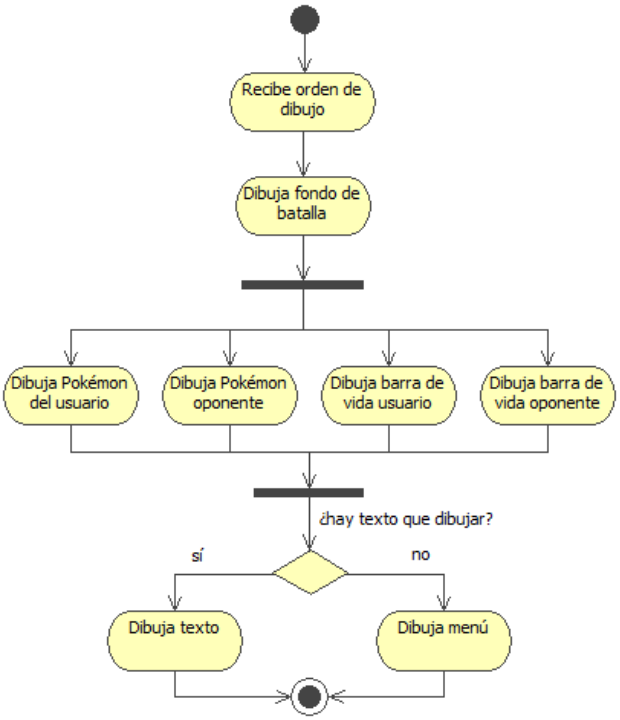

Figura 7.9. Diagrama de actividad para la función de dibujar el combate

En dicho diagrama se puede observar cómo se paralelizan las tareas de dibujar los Pokémon combatientes y sus barras de vida. Efectivamente, sólo es necesario dibujar en orden el fondo, una vez eso está dibujado, es indiferente qué textura se dibuje después. Una vez eso está dibujado, se sincronizan los hilos y, en función de si hay texto o no, se dibuja el menú de combate o el texto.

7.2. Detalles de implementación

Generar encuentro con un Pokémon salvaje

A continuación se muestra la parte del código que genera aleatoriamente encuentros Pokémon en celdas de captura de tierra.

```
int totalRate = 0;
switch (cell->getCaptureZoneType()){
case CaptureZoneList::Land:
	//  Generamos un valor aleatorio sobre 100
	int r = rand() % 100;
	//  Para Tierra, existe un 11% de probabilidades de encontrar un Pokémon
	if (r <= 11){
	//  Encontramos un Pokémon. Debemos ver de qué especie es
	r = rand() % 100;
	//  El valor generado sobre 100 se corresponderá al de la especie
	for (int i = 0; i < map->getNofLandPokemon(); i++){
		totalRate += map->getLandPokemon()[i]->getOccurrenceRatio();
		//  Comprobamos si el valor aleatorio corresponde a esta especie de
pokémon salvaje
		if (r < totalRate){
		//  Cambiamos la vista
		controller->changeView(ViewList::BattleView);

		//  Inicializamos la batalla
		controller->getBattleController()->initWildBattle(player-
>getPokemon(), getWildPokemon(map->getLandPokemon()[i]));

		return;
		}
	}
	}
	break;
}
```

Cuando se encuentra un Pokémon salvaje se llama a la función que obtiene el Pokémon salvaje:

```
Pokemon* WorldController::getWildPokemon(WildPokemon* wp){
	iSpeciesDict* isp = SpeciesDict::getInstance();
	Species* s = isp->getSpecies(wp->getSpeciesNum());

	//  Obtenemos el nombre de la especie conforme al idioma actual
	s->setName(controller->getTalker()->getPokemonName(wp->getSpeciesNum()));

	//  Si encontramos al pokémon creamos su instancia
	Pokemon* p = new Pokemon(s);
	PokemonConstructor* pc = new PokemonConstructor(p);

	//  Calculamos el nivel del pokémon
	int level = (rand() % (wp->getMaxLevel() - wp->getMinLevel() + 1)) + wp-
>getMinLevel();
```

```
        //  Inicializamos el Pokemon salvaje con el nivel calculado
        pc->initPokemon(level);

        //  Calculamos sus movimientos en función de su nuevo nivel
        pc->calculateMoveset();

        delete pc;
        return p;

}
```

Paralelizar al dibujar combate

Para paralelizar el dibujado del combate se utiliza, de nuevo, OpenMP, sin embargo, las funciones utilizadas son diferentes. En este caso, se divide el código a paralelizar en diferentes secciones, bloques de instrucciones que ejecuta un hilo exclusivamente. Es decir, se asigna un hilo a cada sección mediante el uso de:

```
#pragma omp sections.
```

Las secciones se han de declarar dentro de un bloque de secciones. En este caso, el resultado será el siguiente:

```
#pragma omp parallel
        {
        //      Dibujamos el fondo
#pragma omp sections
        {
#pragma omp section
        //      Dibujamos el pokemon del usuario
#pragma omp section
        //      Dibujamos el pokemon rival
#pragma omp section
        //      Dibujamos la barra de vida del pokemon del usuario
#pragma omp section
        //      Dibujamos la barra de vida del pokemon rival
        }
        }
        //      Si hay un texto que mostrar, lo mostramos y sino dibujamos el menú
}
```

7.3. Pruebas

De nuevo, la especificación de las pruebas unitarias utilizará la plantilla de la Tabla 5.25 y mostrará los casos de prueba para la validación de este módulo.

TC-18	Huir de un combate Pokémon
Requisitos asociados	• UC-BS-01 *Huir del combate* • FRQ-11 *Calcular huida*
Propósito	Comprobar que el sistema permite intentar escapar de un combate y que, dependiendo del resultado del intento, lo consiga o no
Estado	Terminado
Acciones	1. Verificar que el sistema permite seleccionar la opción de *Huida* en el menú del combate 2. Verificar que el sistema permite escapar del combate cuando el resultado del cálculo es positivo

BIBLIOGRAFÍA

Todas las referencias bibliográficas electrónicas han sido consultadas a fecha de 3 de julio de 2014.

[BOE13] Boletín Oficial del Estado. Número 256. Sección III. Página 86829. Disponible en http://www.boe.es/buscar/doc.php?id=BOE-A-2013-11199.

[BULWeb] Bulbapedia. Disponible en http://bulbapedia.bulbagarden.net/wiki/Main_Page

[FS99] Martin Fowler y Kendall Scott. *UML gota a gota*. Addison Wesley Longman, México, 1999. ISBN: 968-444-364-1.

[GHJV94] Erich Gamma, Richard Helm, Ralph Johnson y John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*, First Edition. Addison-Wesley, USA, 1994. ISBN: 0-201-63361-2.

[IBM97] IBM OOTC. *Developing Object–Oriented Software*. IBM Object–Oriented Technology Center. Prentice–Hall, 1997.

[IE830] IEEE Computer Society. *IEEE Std. 830-1998, IEEE Recommended Practice for Software Requirements Specifications*. IEEE, 1998. ISBN: 0-7381-0332-2.

[INT08] Instituto Nacional de Tecnologías de la Comunicación. *Guía práctica de gestión de la configuración*. 2008. Disponible en https://www.inteco.es/file/NRDmviQoTbItSH7Apl5n5Q.

[ISO11] ISO/IEC 25010:2011. *Systems and software engineering – Systems and Software Quality Requirements and Evaluation – System and software quality models*. 2011.

[Nielsen] Jakob Nielsen. *10 Usability Heuristics for User Interface Design*. Disponible en http://www.nngroup.com/articles/ten-usability-heuristics.

[PMI08] Project Management Institute. Guía de los Fundamentos de la Dirección de Proyectos. Cuarta Edición. USA, 2008. ISBN: 978-1-933890-72-2.

[PXWeb] Pokexperto. Disponible en http://www.pokexperto.net/.

[SWE99] IEEE Computer Society. *SWEBOK: Guide to the Software Engineering Body of Knowledge*, Versión 3.0. IEEE, 1999. ISBN-10: 0-7695-5166-1. Disponible en http://www.swebok.org.

[WDWeb] WikiDex. Disponible en http://es.pokemon.wikia.com/wiki/WikiDex.